

Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство



Семинарски рад из предмета
Напредне Јава Технологије

Веб апликација за изнајмљивање видео конзола

Ментор: **проф. Др. Милош Милић**

Студент: **Алекса Секулић**

Број индекса: **2022/0171**

Београд, 2025.

Contents

1.	Вербални опис	3
1.1	Основни циљеви.....	3
1.2	Типови корисника.....	3
1.3	Функционалности	4
1.4	Планирани случајеви коришћења (Use Case).....	4
1.5	Архитектура и технологије	5
2.	Технологије	6
2.1.	Spring Boot	6
2.2.	Spring Boot Web.....	6
2.3.	Spring Data JPA.....	7
2.4.	ReactJS.....	7
2.5.	Maven	8
3.	Случајеви коришћења.....	9
4.	Пројектовање.....	10
	СК2 – Пријава клијента	13
	СК3 – Креирање клијента	15
	СК4 – Регистрација клијента	18
	СК5 – Брисање клијента.....	20
	СК6 – Претрага клијента	23
	СК7 – Измена клијента	24
	СК8 – Креирање конзоле	26
	СК9 – Брисање конзоле.....	29
	СК10 – Претрага конзоле	31
	СК11 – Измена конзоле	32
	СК12 – Креирање изнајмљивања	34
	СК13 – Измена изнајмљивања	36
	СК14 – Претрага изнајмљивања.....	39
5.	Структура пројекта	40
5.1	Структура система	41
6.	Закључак	46

1. Вербални опис

Пројекат „Изнајмљивање видео конзола“ представља модерну веб апликацију за управљање процесом изнајмљивања конзола, као што су **PlayStation 5**, **Xbox** и друга пратећа опрема (џојстици, слушалице, VR уређаји, итд).

Апликација је заснована на **клијент–сервер архитектури**, где је **позадински део (backend)** имплементиран у **Java Spring Boot** радном оквиру, док је **кориснички интерфејс (frontend)** развијен у **ReactJS** технологији.

Циљ система је дигитализација и аутоматизација процеса изнајмљивања видео-опреме, уз могућност праћења залиха, статуса опреме и историје изнајмљивања.

1.1 Основни циљеви

Главни циљ пројекта је да поједностави комуникацију између клијената и радника (администратора) који управљају системом.

Специфични циљеви обухватају:

- **За клијенте:**
 - Једноставан приступ листи доступних конзола и додатне опреме.
 - Регистрацију и пријаву на систем.
 - Преглед статуса својих изнајмљивања.
- **За раднике (администраторе):**
 - Централизовано управљање свим клијентима и опремом.
 - Преглед, измена и брисање ставки у систему (CRUD операције).
 - Контрола статуса изнајмљивања (у току, завршено, отказано).
 - Потписивање и евиденција уговора о изнајмљивању.

1.2 Типови корисника

Систем подржава два типа корисника:

- **Клијент:** може да претражује понуду, прегледа историју и статусе.
- **Радник:** има приступ свим клијентским функционалностима уз додатне административне алате за управљање.

1.3 Функционалности

Нерегистровани корисници:

- Регистрација новог клијента.

Регистровани клијенти:

- Претрага конзола и додатне опреме.
- Преглед и праћење статуса (у току, завршено, отказано).
- Одјава са налога.
- Уплаћивање кредита.

Радници (администратори):

- **Управљање клијентима (CRUD):** додавање, измена, брисање, претрага.
- **Управљање конзолама (CRUD):** додавање нових, измена података, брисање, преглед стања (слободна, заузета, сервис).
- **Управљање изнајмљивањима:** креирање, измена, брисање и преглед свих активних и архивираних изнајмљивања.
- **Контрола статуса и плаћања:** евиденција плаћених и неплаћених изнајмљивања.

1.4 Планирани случајеви коришћења (Use Case)

- **SK1:** Пријави радника
- **SK2:** Пријави клијента
- **SK3:** Креирај клијента
- **SK4:** Регистрација клијента
- **SK5:** Обриши клијента
- **SK6:** Претражи клијента
- **SK7:** Измени клијента
- **SK8:** Креирај конзолу
- **SK9:** Обриши конзолу
- **SK10:** Претражи конзолу
- **SK11:** Измени конзолу
- **SK12:** Креирај изнајмљивање
- **SK13:** Измени изнајмљивање
- **SK14:** Претражи изнајмљивање

1.5 Архитектура и технологије

Backend (Позадински део):

- **Језик:** Java
- **Радни оквир:** Spring Boot
- **Пристап бази:** Spring Data JPA, Hibernate
- **База података:** MySQL
- **Архитектура:** REST API

Frontend (Кориснички интерфејс):

- **Језик:** JavaScript / TypeScript
- **Радни оквир:** ReactJS
- **Структура:** Компонентно-оријентисана архитектура са сервисима за позиве према backend API-ју
- **Додатно:** HTML5, CSS3, Bootstrap

Веб апликација „**Изнајмљивање видео конзола**“ представља потпуно функционално решење за модерно управљање процесом изнајмљивања електронске опреме.

Захваљујући комбинацији **ReactJS** интерфејса и **Spring Boot** backend-a, систем нуди брзину, стабилност и лако одржавање.

Даље надоградње могу обухватити интеграцију система за **онлајн плаћање**, **аутоматску нотификацију о истеку изнајмљивања**, или **AI препоруке** за избор опреме на основу историје коришћења.

2. Технологије

2.1. Spring Boot

Spring Boot је оквир (framework) заснован на **Јави**, намењен развоју савремених backend апликација. Његова највећа предност је што знатно поједностављује конфигурацију и покретање апликација у односу на класични Spring Framework.

Захваљујући аутоматској конфигурацији и „starter“ модулима, програмерима није потребно ручно подешавање великог броја конфигурационих фајлова.

Spring Boot омогућава лаку изградњу **REST API** сервиса који служе за комуникацију између frontend-a и backend-a.

У овом пројекту, Spring Boot је основа backend дела апликације и користи се у комбинацији са **Spring Data JPA** за рад са **MySQL** базом података. На тај начин се обезбеђује ефикасно управљање ентитетима као што су *Клијент*, *Изнајмљивање*, *Опрема* и њихово складиштење у бази.

2.2. Spring Boot Web

Spring Boot Web је модул који омогућава развој веб апликација и **REST API** сервиса.

Он омогућава једноставно дефинисање **контролера** који обрађују HTTP захтеве као што су **GET, POST, PUT, DELETE**, чиме се успоставља комуникација између React frontend-a и backend-a.

Spring Boot Web користи **уграђени сервер (Tomcat)**, па није потребно ручно подешавање спољног сервера. Апликација се покреће као стандардна Java апликација, што знатно убрзава развој, тестирање и деплој.

У овом пројекту Spring Boot Web омогућава:

- креирање REST контролера за ентитете као што су *Клијент*, *Конзола*, *Додатна опрема*, *Изнајмљивање*;
- обраду захтева које шаље React frontend;
- слање одговора у JSON формату;
- управљање основним рутама и endpoint-има backend апликације.

2.3. Spring Data JPA

Spring Data JPA је модул који поједностављује рад са релационим базама података. Заснован је на **JPA (Java Persistence API)** спецификацији и омогућава мапирање Java класа (ентитета) на табеле у бази.

Основна предност је што омогућава рад са подацима без писања великих количина SQL упита. Довољно је дефинисати **ентитете** и **repository интерфејсе**, а Spring аутоматски генерише имплементације за најчешће операције као што су:

- чување (save)
- ажурирање (update)
- брисање (delete)
- претрага (find, findById, findByNaziv итд.)

У овом пројекту Spring Data JPA се користи за:

- повезивање апликације са **MySQL** базом података;
- управљање ентитетима као што су *Клијент*, *Радник*, *Опрема*, *Изнајмљивање*, *СтавкаИзнајмљивања*;
- чување и приступ подацима без потребе за ручним SQL упитима;
- дефинисање односа између ентитета (нпр. један-на-више, више-на-један).

2.4. ReactJS

ReactJS је модерна **frontend библиотека** отвореног кода коју је развио **Facebook**, а данас је једно од најпопуларнијих решења за развој динамичких веб апликација.

React омогућава креирање **Single Page Application (SPA)** - апликација које се освежавају без поновног учитавања целе странице, што обезбеђује брзо и глатко корисничко искуство.

React се заснива на **компонентама**, које представљају независне делове интерфејса (нпр. форма за пријаву, табела са клијентима, страница са опремом). Свака компонента управља својим стањем и може да комуницира са другим компонентама преко својстава (props).

У овом пројекту ReactJS се користи за:

- израду корисничког интерфејса за клијенте и раднике;
- повезивање са backend-ом путем **Axios** или **Fetch API** позива;
- имплементацију рутера (React Router) за навигацију између страница (нпр. */login*, */admin*, */app*);
- управљање формама, валидацију уноса и динамичко приказивање података из базе;
- примену модерног CSS дизајна и анимација ради бољег UX-а.

2.5. Maven

Maven је алат за управљање пројектом и аутоматизацију изградње, најчешће коришћен у **Java** екосистему.

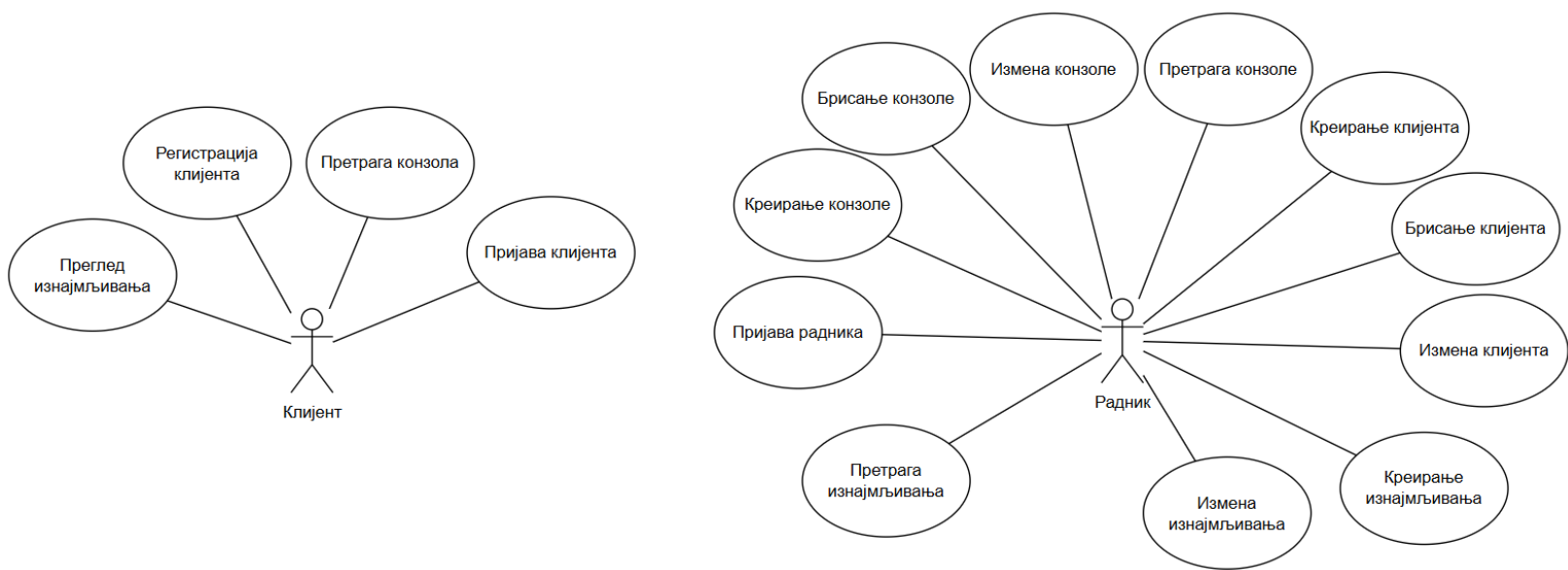
У свом основном документу, **pom.xml**, дефинишу се све зависности, plug-in-ови и конфигурације које су потребне за рад и изградњу апликације.

Maven програмерима омогућава да лако укључе спољне библиотеке без ручног преузимања. Он аутоматски преузима зависности, брине о компилацији, тестирању и паковању апликације у извршив формат (JAR/WAR).

У овом пројекту Maven служи за:

- управљање зависностима као што су Spring Boot, Spring Data JPA, Lombok, MySQL Driver;
- изградњу и покретање backend дела апликације;
- поједностављен процес развоја, компилације и деплоја Spring Boot сервиса.

3. Случајеви коришћења



Слика 1: Случајеви коришћења

За следеће случајеве коришћења ћемо дати детаљан опис:

Ознака	Назив случаја коришћења
СК1	Пријава радника
СК2	Пријава клијента
СК3	Креирање клијента
СК4	Регистрација клијента
СК5	Брисање клијента
СК6	Претрага клијента
СК7	Имена клијента
СК8	Креирање конзоле
СК9	Брисање конзоле
СК10	Претрага конзоле
СК11	Имена конзоле
СК12	Креирање изнајмљивања
СК13	Имена изнајмљивања
СК14	Претрага изнајмљивања

Табела 1: Табела случајева коришћења

4. Пројектовање

СК1 – Пријава радника

Назив СК:

Пријава радника

Актори СК:

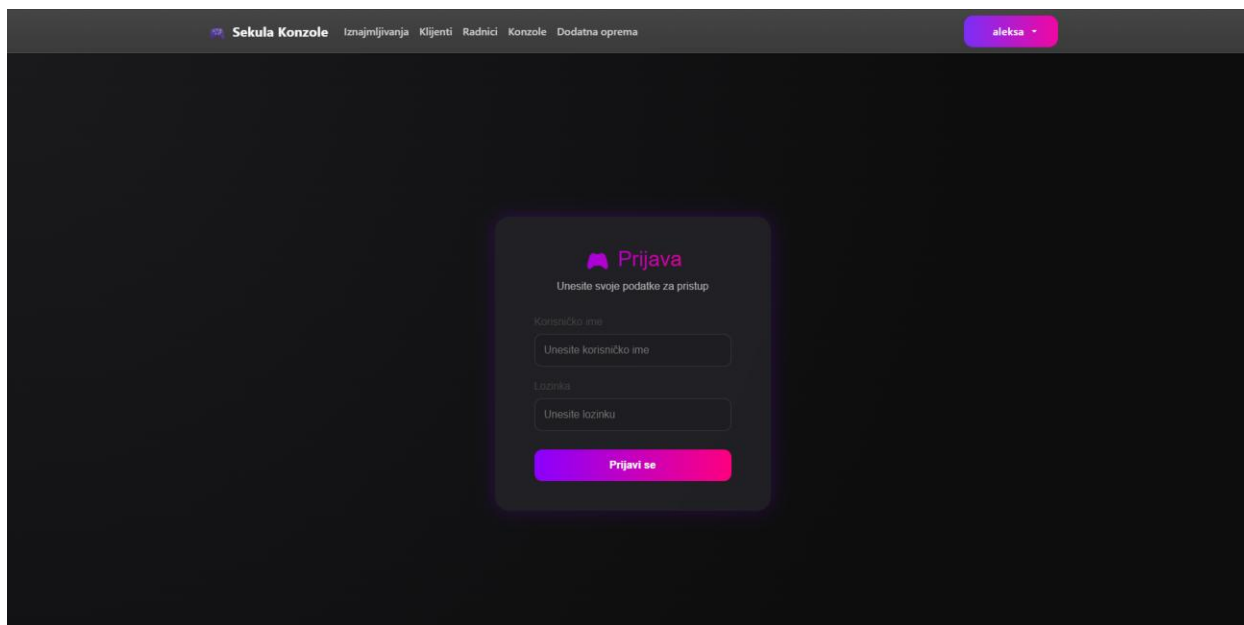
Радник

Учесници СК:

Радник, кориснички интерфејс (React апликација), систем (Spring Boot сервер)

Предуслови:

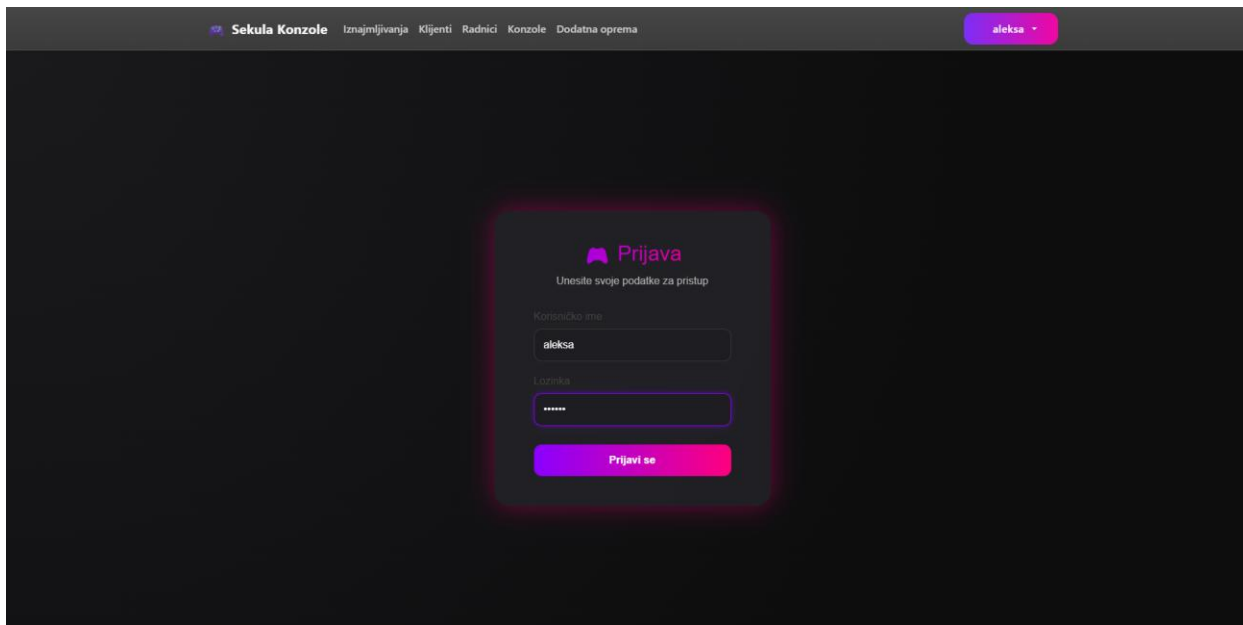
Клијентски и серверски програми су покренути. Радник има регистрован налог.



Слика 2: Екран за пријаву

Основни сценарио СК:

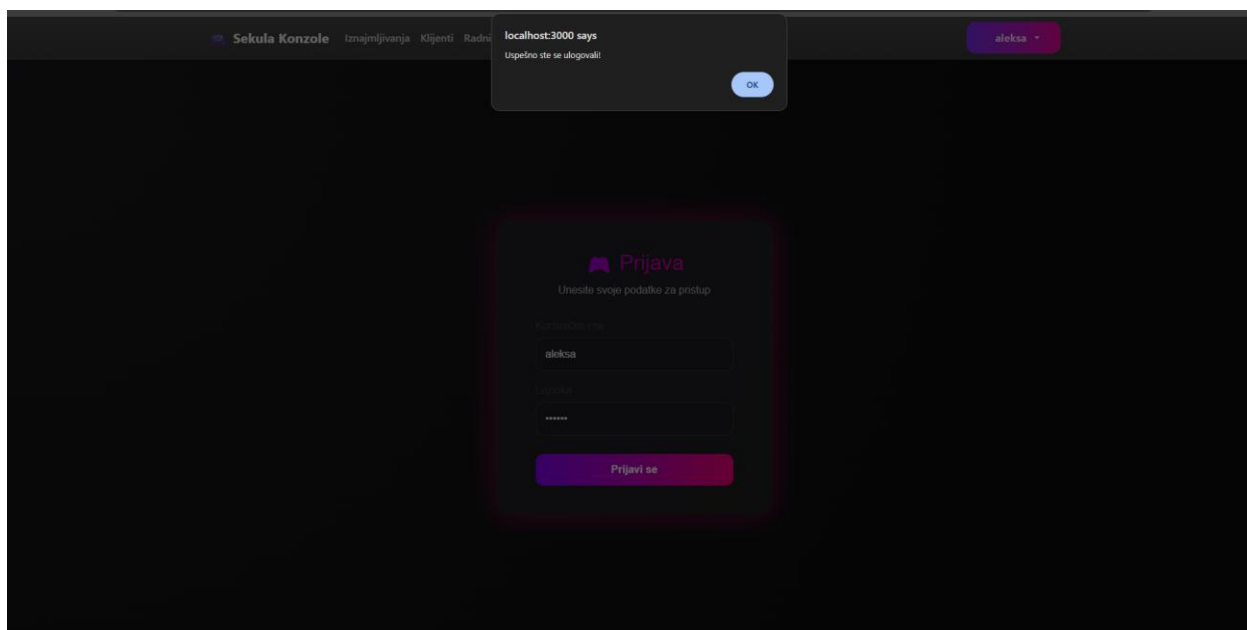
1. Радник уноси корисничко име и лозинку. (АПУСО)



The screenshot shows the 'Prijava' (Login) form in the 'Sekula Konzole' application. The form is centered on a dark background. It has a title 'Prijava' with a key icon, followed by the instruction 'Unesite svoje podatke za pristup'. There are two input fields: 'Korisničko ime' (Username) containing 'aleksa' and 'Lozinka' (Password) containing '*****'. A red 'Prijava se' button is at the bottom of the form. The top navigation bar includes links for 'Sekula Konzole', 'Iznajmljivanje', 'Klijenti', 'Radnici', 'Konzole', and 'Dodatna oprema'. A user profile dropdown with the name 'aleksa' is in the top right corner.

Слика 3: Форма за пријаву са попуњеним подацима

2. Радник позива систем за пријаву. (АПСО)
3. Систем проверава податке. (СО)
4. Систем приказује поруку: „Успешно сте се улоговали!“ (ИА)

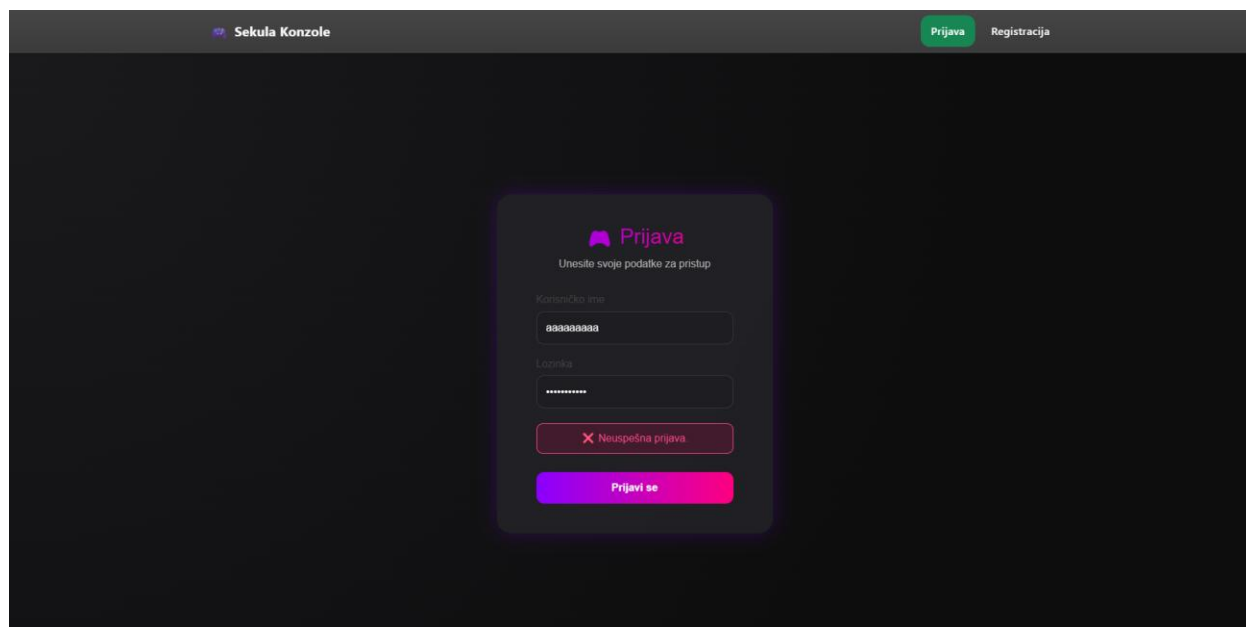


This screenshot is similar to the previous one, showing the 'Prijava' form. However, a modal message box is overlaid at the top center. The message box has a title 'localhost:3000 says' and the text 'Успешно сте се улоговали!' (Successfully logged in!). It includes an 'OK' button. The login form and navigation bar are visible in the background.

Слика 4: Порука успеха

Алтернативна сценарија:

4.1 Ако подаци нису валидни, систем приказује поруку: „Неуспешна пријава.“ (ИА)



The screenshot displays the login interface of the 'Sekula Konzole' system. The header features the application name and navigation links for login and registration. The main content area contains a login form with fields for username and password, which are currently masked. A red error message indicates a failed login attempt. A blue button is provided for the user to attempt login again.

Слика 5: Први алтернативни сценарио СК Пријави радника

СК2 – Пријава клијента

Назив СК:

Пријава клијента

Актори СК:

Клијент

Учесници СК:

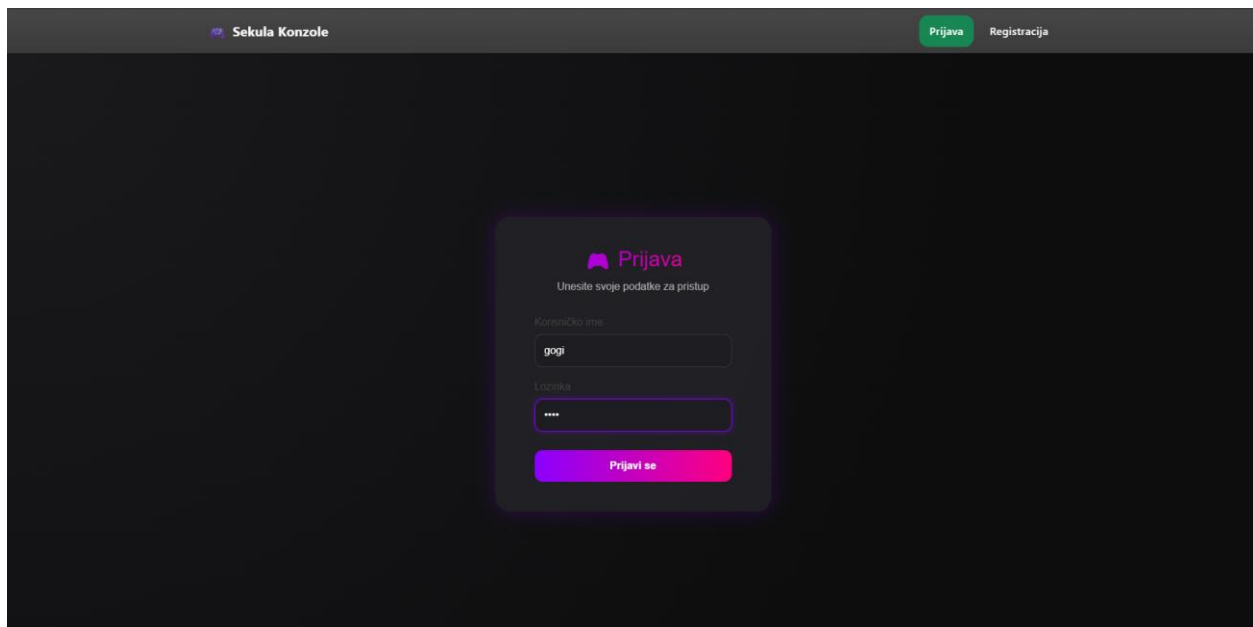
Клијент, кориснички интерфејс (React), систем (Spring Boot)

Предуслови:

Клијент и сервер су покренути. Клијент има регистрован налог.

Основни сценарио СК:

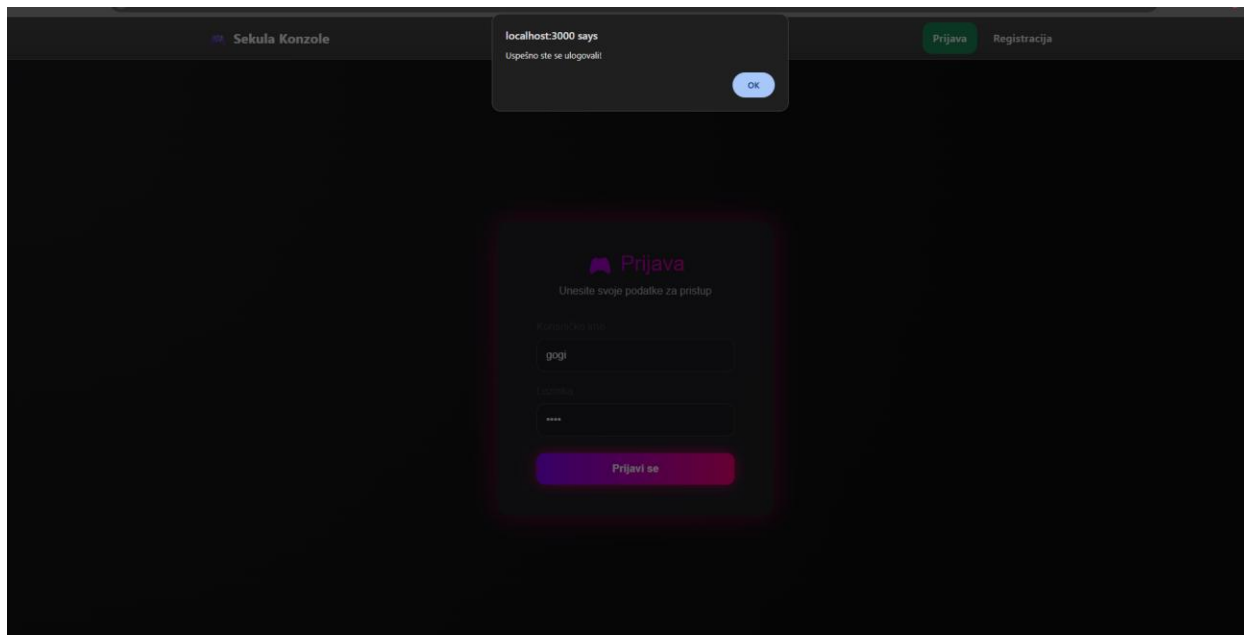
1. Клијент уноси своје корисничко име и лозинку. (АПУСО)



The screenshot shows a web application interface for 'Sekula Konzole'. At the top, there is a dark header bar with the text 'Sekula Konzole' on the left and two buttons, 'Prijava' (Login) and 'Registracija' (Registration), on the right. The 'Prijava' button is highlighted in green. The main content area has a dark background. In the center, there is a light gray rounded rectangle containing the login form. The form has a title 'Prijava' with a key icon, followed by the instruction 'Unesite svoje podatke za pristup'. Below this, there are two input fields: 'Korisničko ime' (Username) with the value 'gogi' and 'Lozinka' (Password) with masked characters '****'. At the bottom of the form is a red button labeled 'Prijavi se' (Login).

Слика 6: Форма пријави клијента са попуњеним подацима

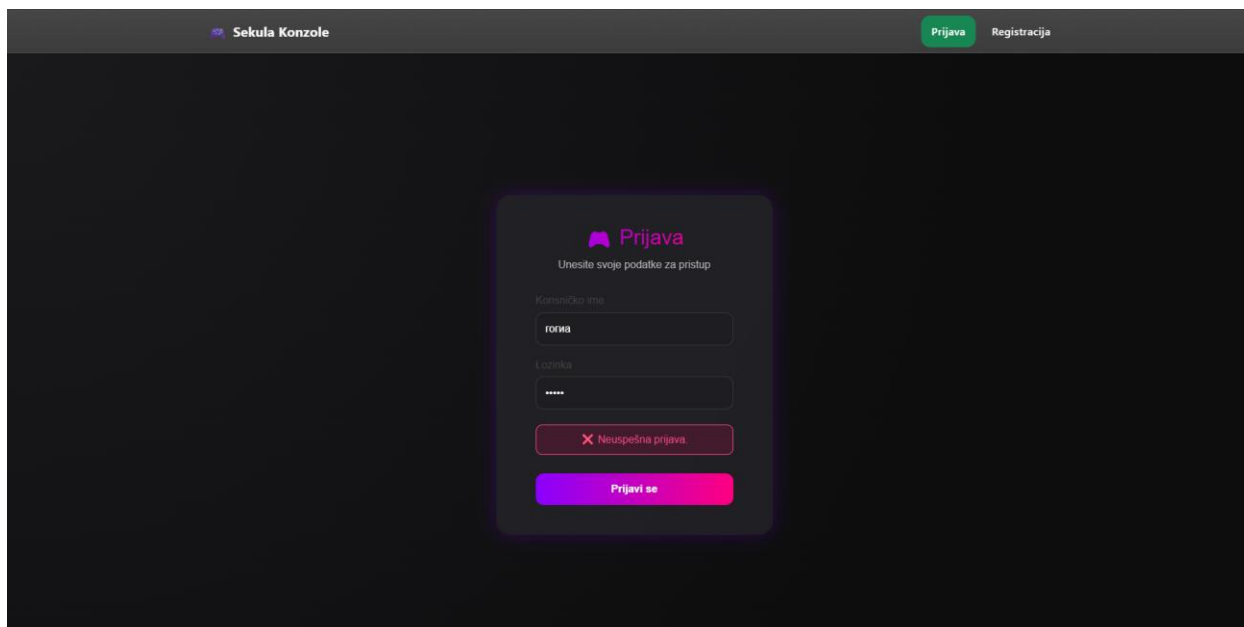
2. Клијент позива систем за пријаву. (АПСО)
3. Систем проверава податке у бази. (СО)
4. Систем приказује поруку: „Успешно сте се улоговали!“ (ИА)



Слика 7: Приказ успешног пријављивања

Алтернативна сценарија:

4.1 Ако подаци нису валидни: „Неуспешна пријава.“ (ИА)



Слика 8: Приказ првог алтернативног сценарија СК Пријави клијента

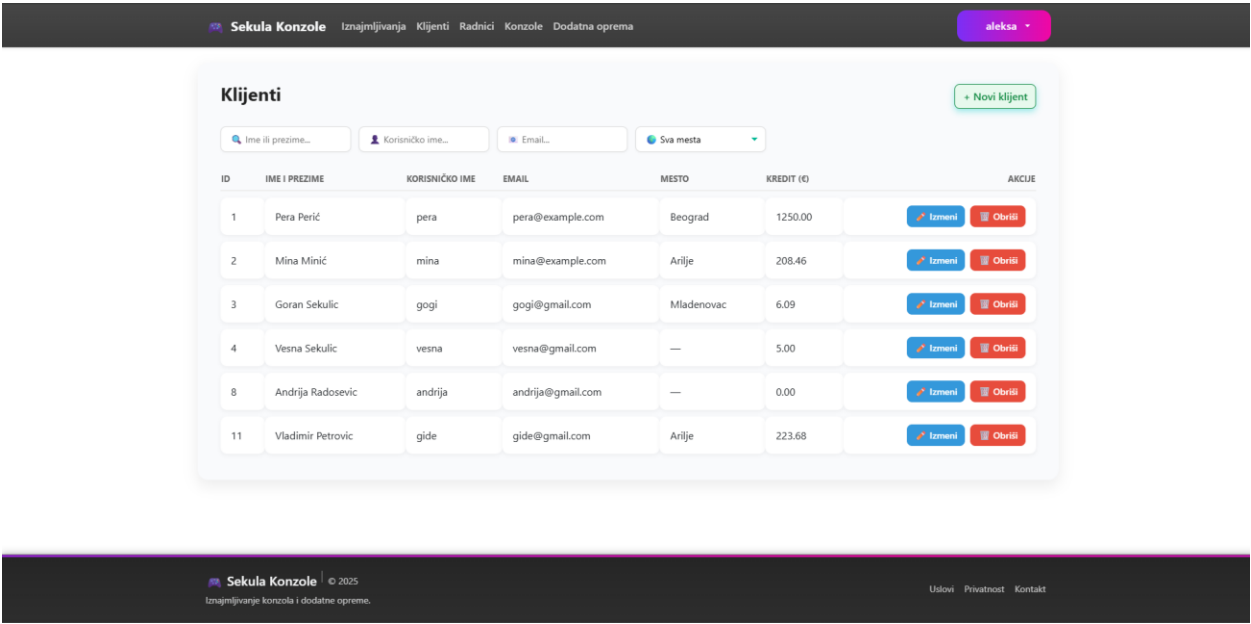
СКЗ – Креирање клијента

Назив СК:
Креирање клијента

Актори СК:
Радник

Учесници СК:
Радник, интерфејс, систем

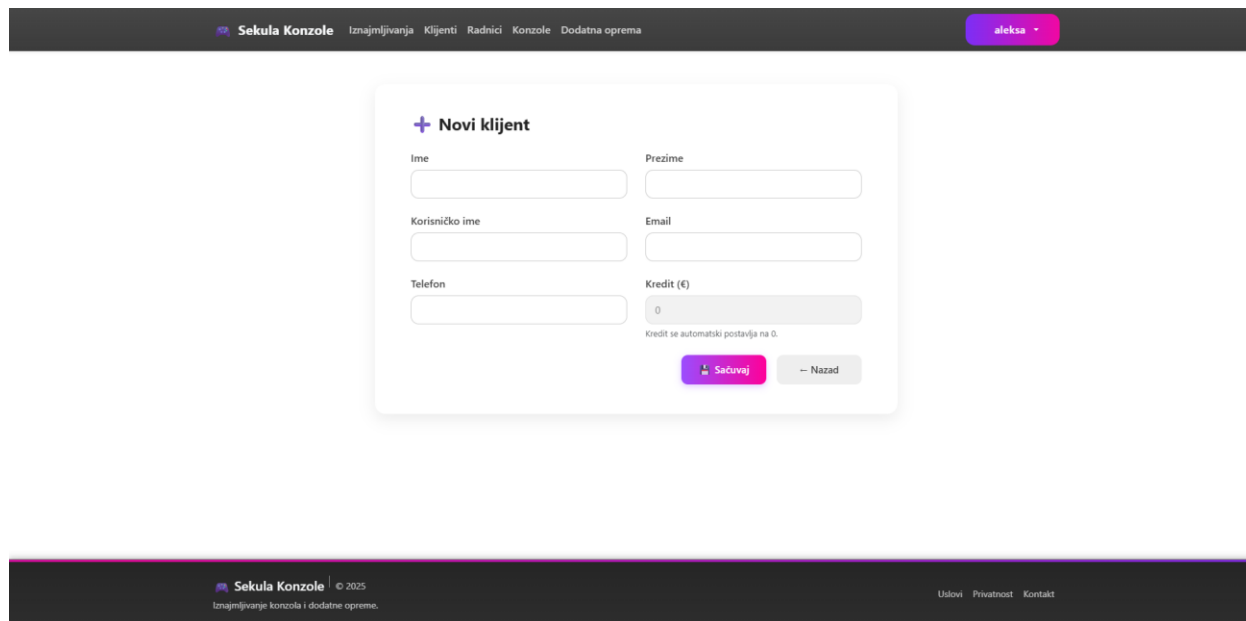
Предуслови:
Радник је пријављен, а клијентски и серверски програми су покренути.



Слика 9: Форма приказ клијената

Основни сценарио СК:

1. Радник позива систем да отвори форму за креирање клијента. (АПСО)
2. Систем приказује форму са пољима (име, презиме, телефон, корисничко име, лозинка, кредит, емаил). (ИА)



Sekula Konzole | Iznajmljivanja | Klijenti | Radnici | Konzole | Dodatna oprema

aleksa

+ Novi klijent

Ime:

Prezime:

Korisničko ime:

Email:

Telefon:

Kredit (€):

Kredit se automatski postavlja na 0.

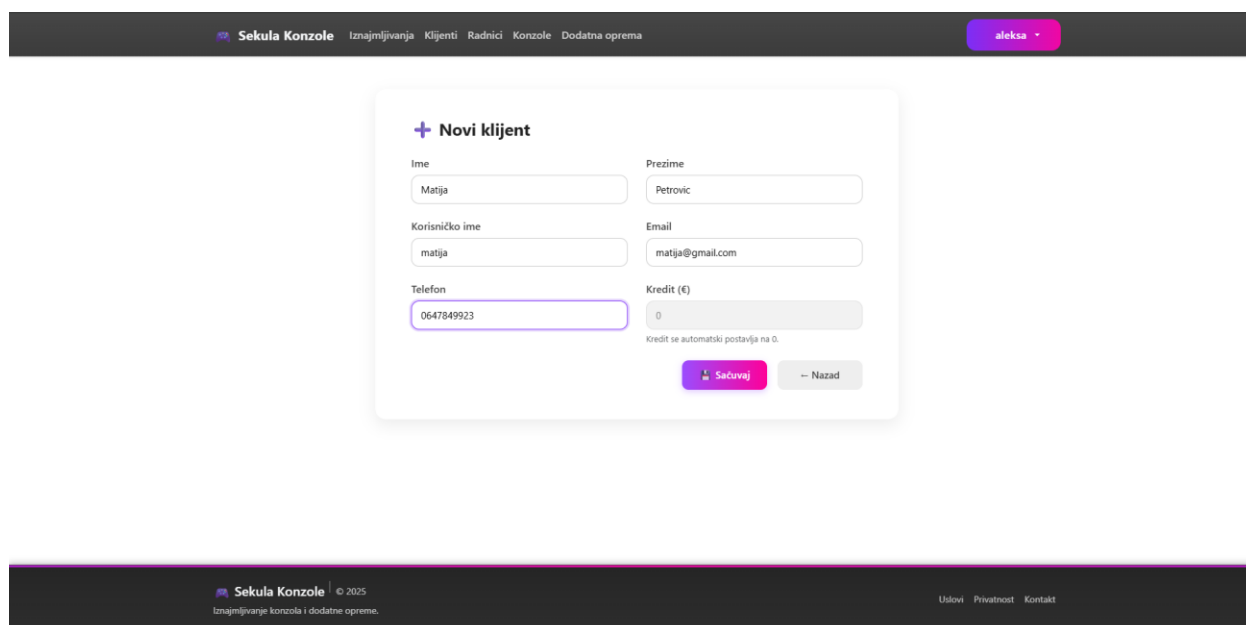
Sačuvaj Nazad

Sekula Konzole | © 2025
Iznajmljivanje konzola i dodatne opreme.

Uслови | Privatnost | Kontakt

Слика 10: Форма Креирај клијента

3. Радник уноси податке. (АПУСО)



Sekula Konzole | Iznajmljivanja | Klijenti | Radnici | Konzole | Dodatna oprema

aleksa

+ Novi klijent

Ime:

Prezime:

Korisničko ime:

Email:

Telefon:

Kredit (€):

Kredit se automatski postavlja na 0.

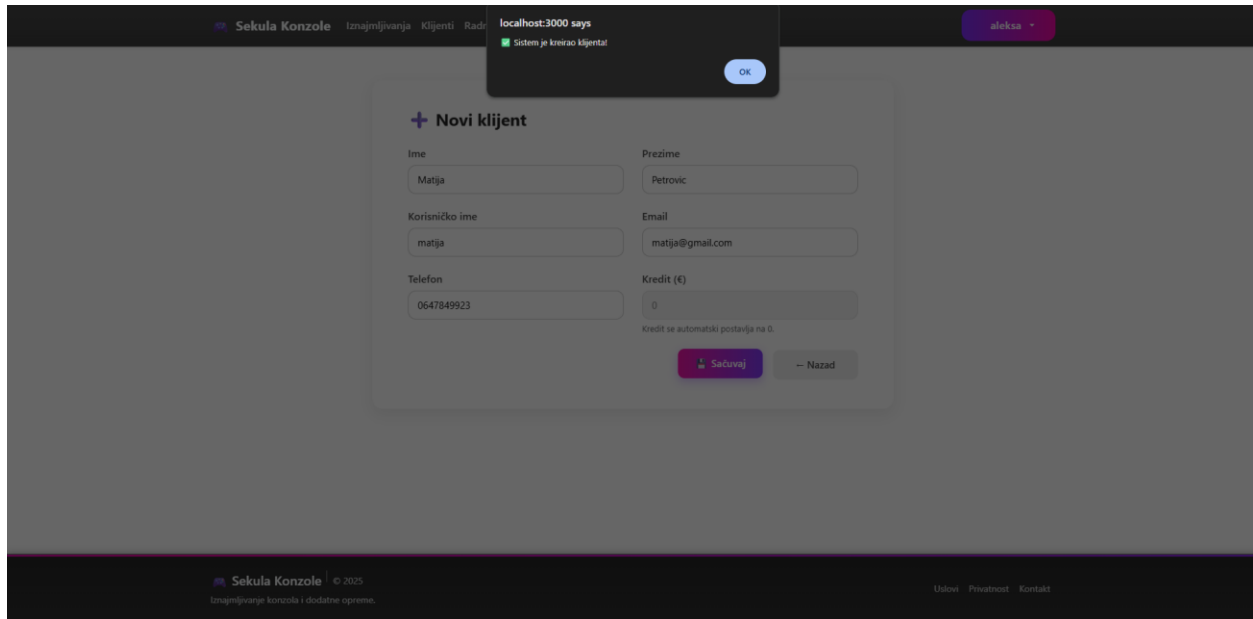
Sačuvaj Nazad

Sekula Konzole | © 2025
Iznajmljivanje konzola i dodatne opreme.

Uслови | Privatnost | Kontakt

Слика 11: Форма Креирај клијента са попуњеним подацима

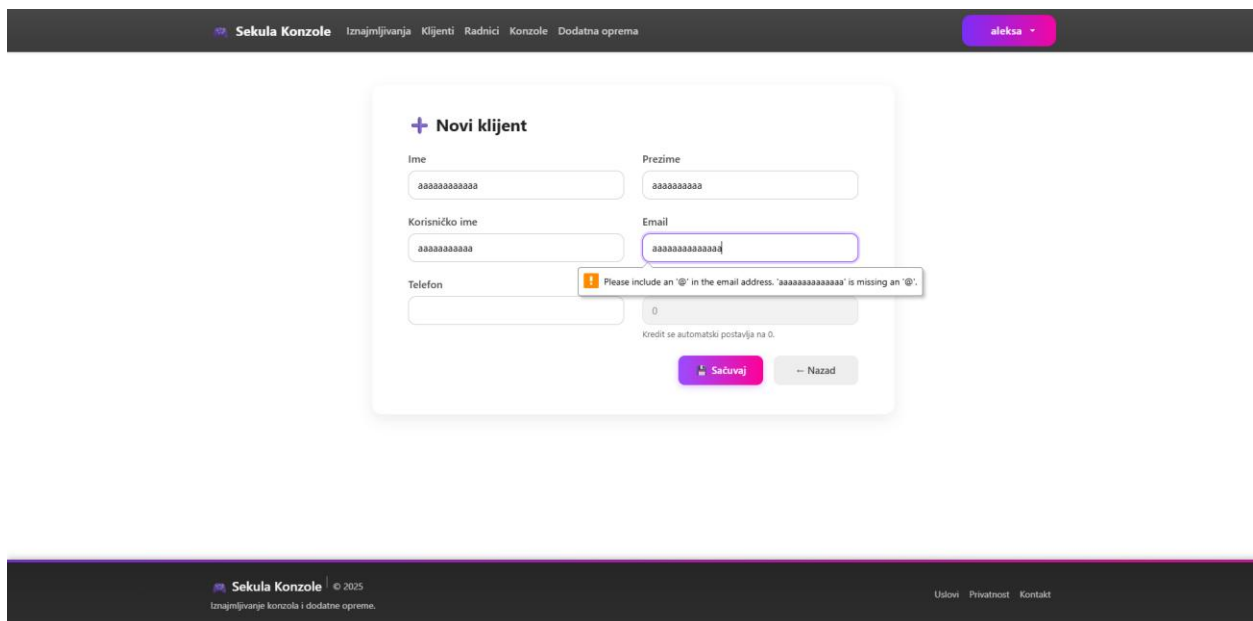
4. Радник позива систем да сачува податке. (АПСО)
5. Систем креира новог клијента у бази. (СО)
6. Систем приказује поруку: „ Sistem je kreirao klijenta!“ (ИА)



Слика 12: Порука успешног креирања клијента

Алтернативна сценарија:

6.1 Ако систем не може да креира клијента, систем исписује грешку: „This client already has an account!“ (ИА)



Слика 13: Приказ првог алтернативног сценарија СК Креирај клијента

СК4 – Регистрација клијента

Назив СК:

Регистрација клијента

Актори СК:

Клијент

Учесници СК:

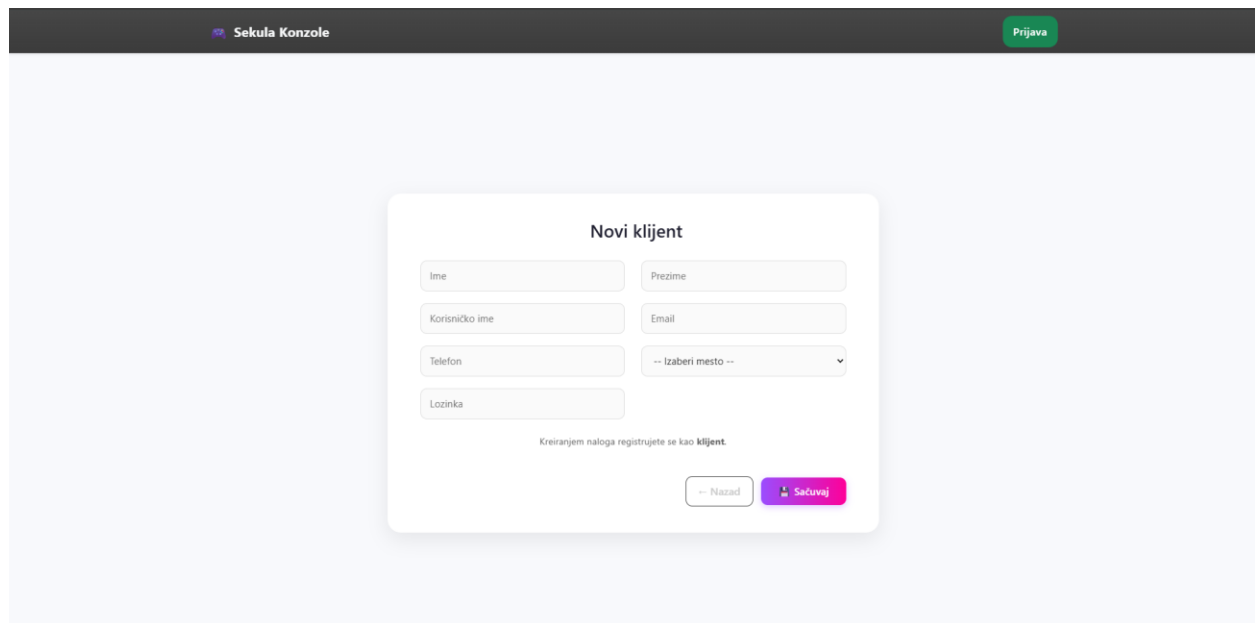
Клијент, интерфејс (React), систем

Предуслови:

Клијент и сервер су покренути.

Основни сценарио СК:

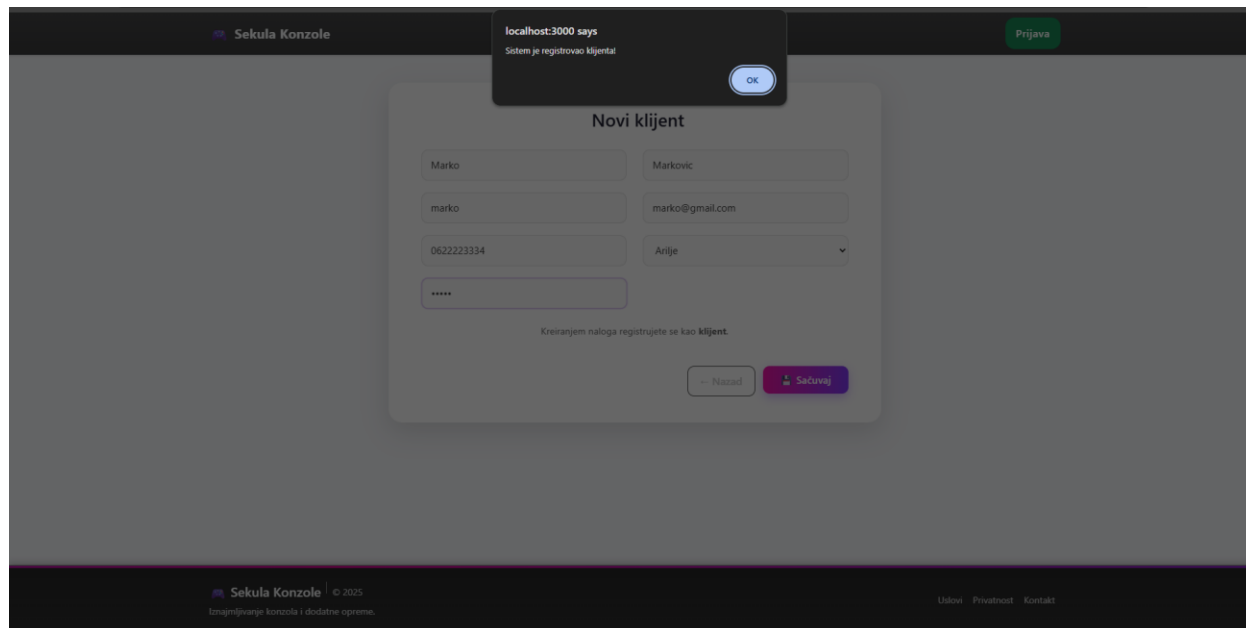
1. Клијент позива систем да отвори форму за регистрацију. (АПСО)
2. Систем приказује форму за унос података. (ИА)



The screenshot shows a web application interface with a dark header bar. On the left, it says 'Sekula Konzole' with a small icon. On the right, there is a green button labeled 'Prijava'. Below the header, the main content area is light gray and contains a white registration form titled 'Novi klijent'. The form has several input fields: 'Ime', 'Prezime', 'Korisničko ime', 'Email', 'Telefon', and 'Lozinka'. There is also a dropdown menu labeled '-- Izaberi mesto --'. Below the inputs, a small text line reads 'Kreiranjem naloga registrujete se kao klijent.'. At the bottom of the form, there are two buttons: a light gray button labeled '-- Nazad' and a purple button labeled 'Sačuvaj' with a small icon of a person.

Слика 14: Приказ форме за регистрацију

3. Клијент уноси податке (име, презиме, телефон, емаил, корисничко име, лозинку). (АПУСО)
4. Клијент потврђује унос. (АНСО)
5. Клијент позива систем да сачува податке. (АПСО)
6. Систем креира нови налог. (СО)
7. Систем приказује поруку: „ Sistem je registrovao klijenta!“ (ИА)



Слика 15: Порука успешног регистравања

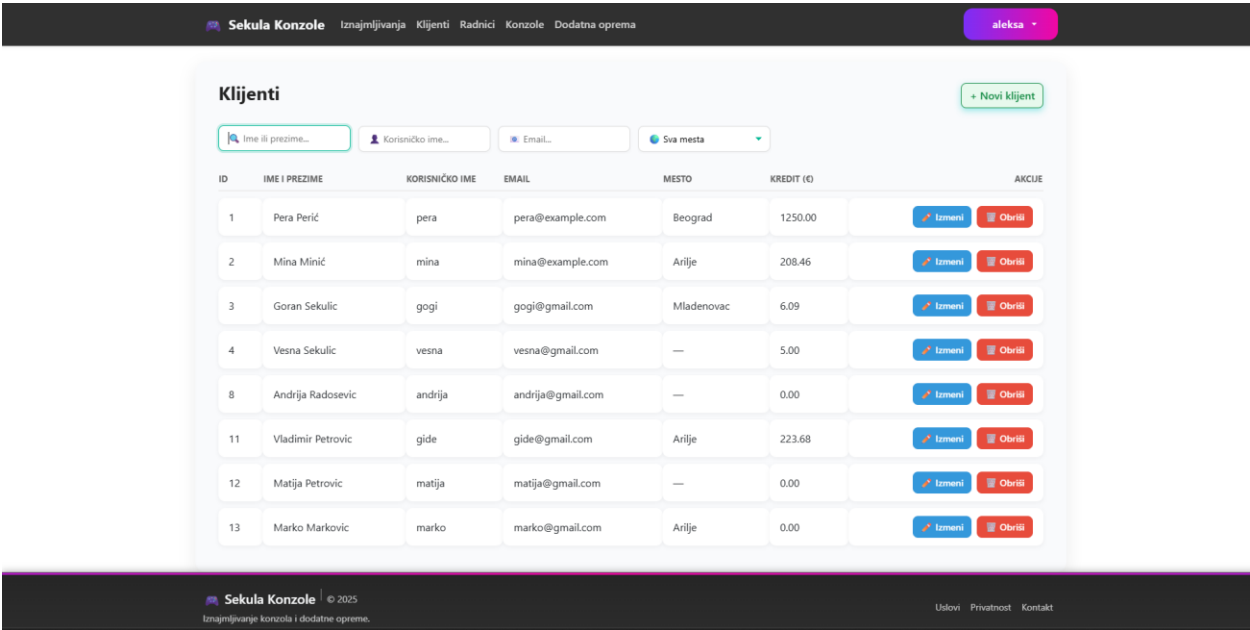
СК5 – Брисање клијента

Назив СК:
Брисање клијента

Актори СК:
Радник

Учесници СК:
Радник, интерфејс, систем

Предуслови:
Радник је пријављен.



Слика 16: Форма приказ клијената

Основни сценарио СК:

1. Радник претражује клијенте. (АПУСО)
2. Систем приказује резултате. (ИА)

Sekula KonzoleIznajmijvanjaKlijentiRadniciKonzoleDodatna oprema

aleksa

Klijenti

+ Novi klijent

marko

Korisničko ime...

Email...

Sva mesta

ID	IME I PREZIME	KORISNIČKO IME	EMAIL	MESTO	KREDIT (€)	AKCIJE
13	Marko Markovic	marko	marko@gmail.com	Arije	0.00	Ispravi Obrisi

Sekula Konzole

© 2025

Iznajmijvanje konzola i dodatne opreme.

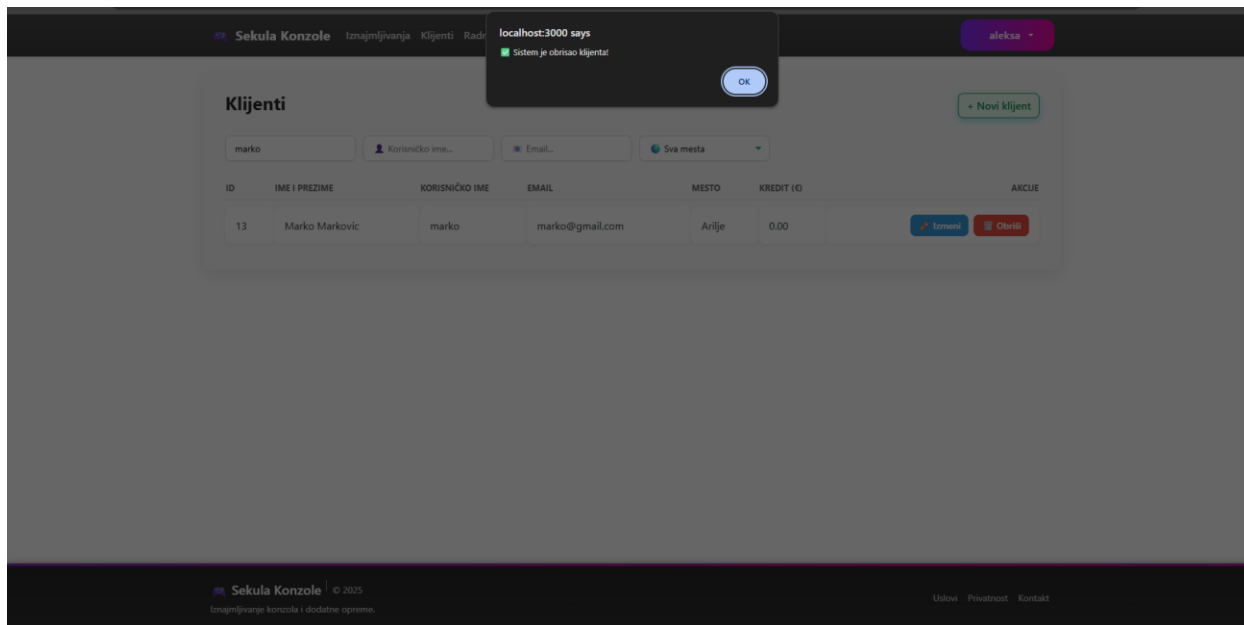
Uslugi

Privatnost

Kontakt

Слика 17: Приказ успешне претраге

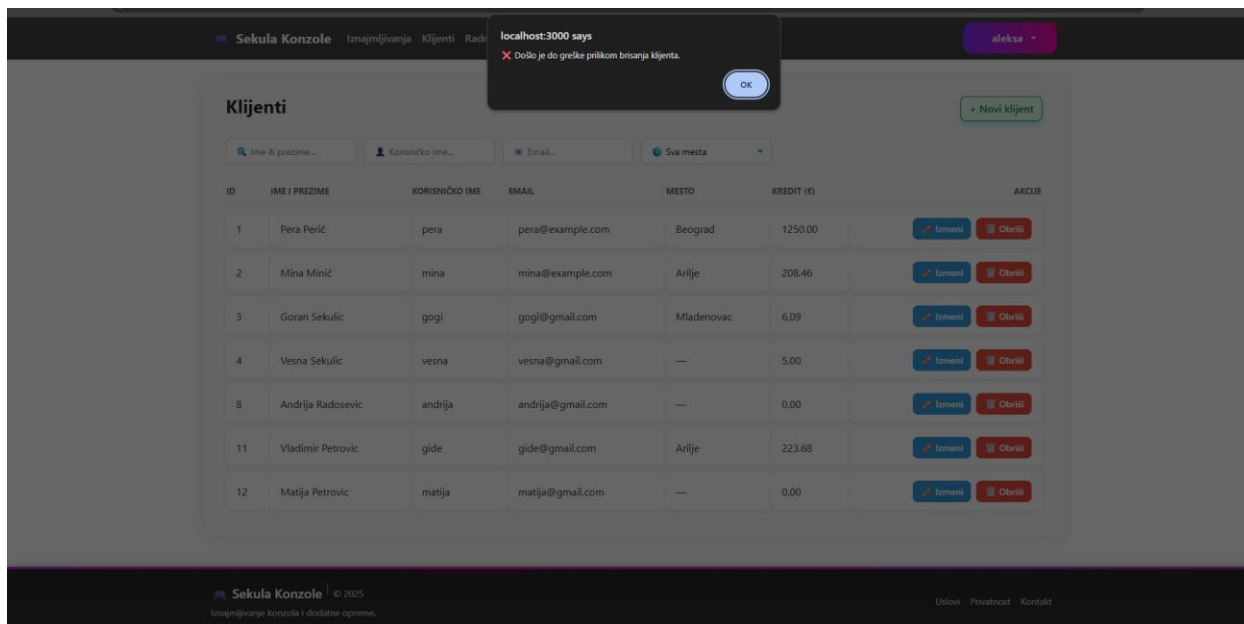
3. Радник бира клијента и позива систем за брисање. (АПСО)
4. Систем брише клијента из базе. (СО)
5. Систем приказује поруку: „Client deleted successfully!“ (ИА)



Слика 18: Приказ успешног брисања клијента

Алтернативна сценарија:

5.1 Ако клијент не може бити обрисан (нпр. активна изнајмљивања): „Došlo je do greške prilikom brisanja klijenta!“ (ИА)



Слика 19: Приказ првог алтернативног сценарија СК Обриши клијента

СК6 – Претрага клијента

Назив СК:

Претрага клијента

Актори СК:

Радник

Учесници СК:

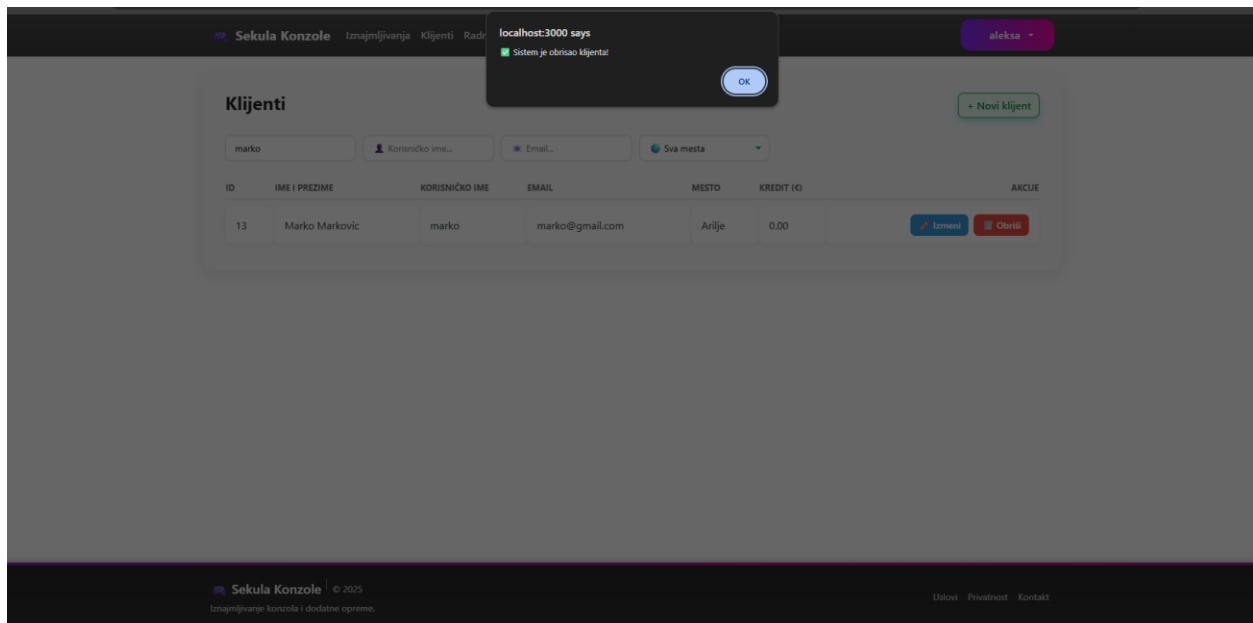
Радник, интерфејс, систем

Предуслови:

Радник је пријављен.

Основни сценарио СК:

1. Радник уноси параметре претраге (име, презиме или корисничко име). (АПУСО)
2. Систем претражује базу клијената. (СО)
3. Систем приказује резултате. (ИА)



Слика 20: Приказ успешне претраге клијента

СК7 – Измена клијента

Назив СК:

Измена клијента

Актори СК:

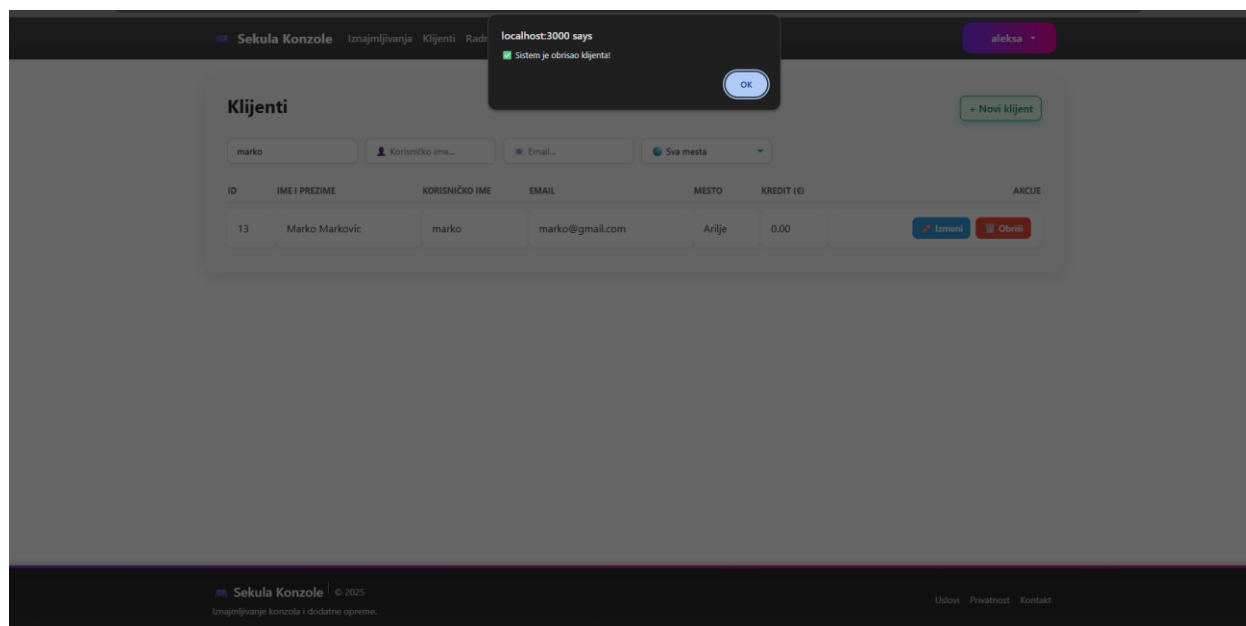
Радник

Учесници СК:

Радник, интерфејс, систем

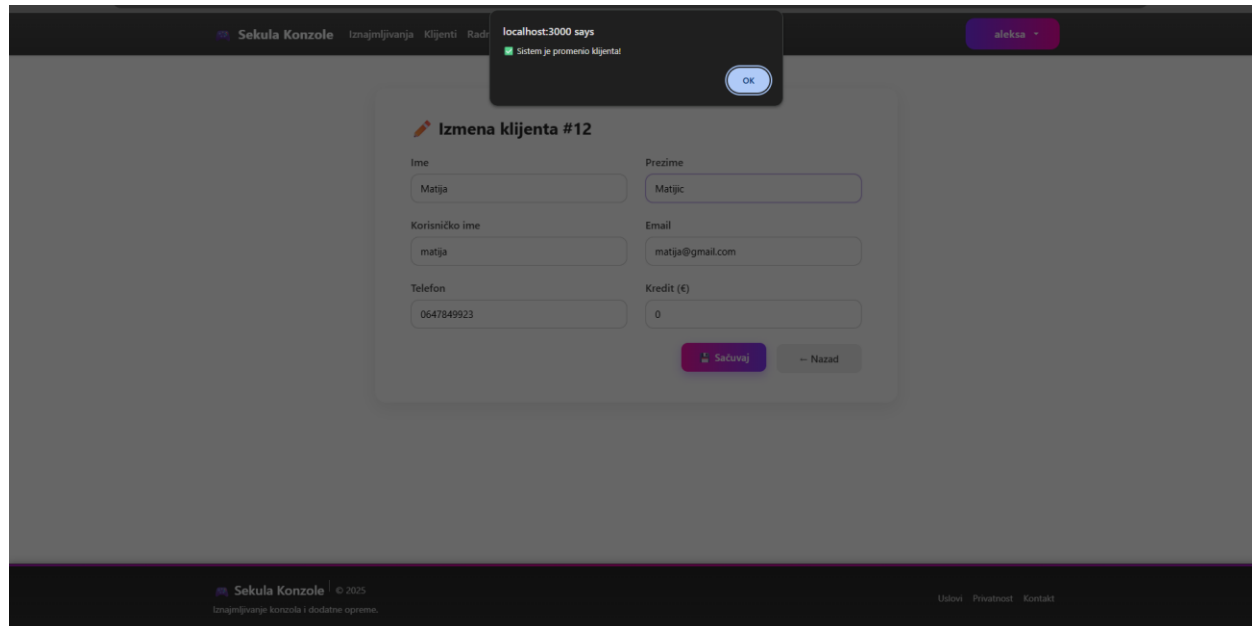
Основни сценарио СК:

1. Радник претражује клијенте. (АПУСО)
2. Систем приказује резултате. (ИА)



Слика 21: Приказ успешне претраге клијента

3. Радник мења податке (име, презиме, телефон, емаил, кредит). (АПУСО)
4. Радник позива систем да сачува измене. (АПСО)
5. Систем чува измене. (СО)
6. Систем приказује поруку: „Sistem je promenio klijenta!“ (ИА)



Слика 22: Порука успешне измене клијента

СК8 – Креирање конзоле

Назив СК:

Креирање конзоле

Актори СК:

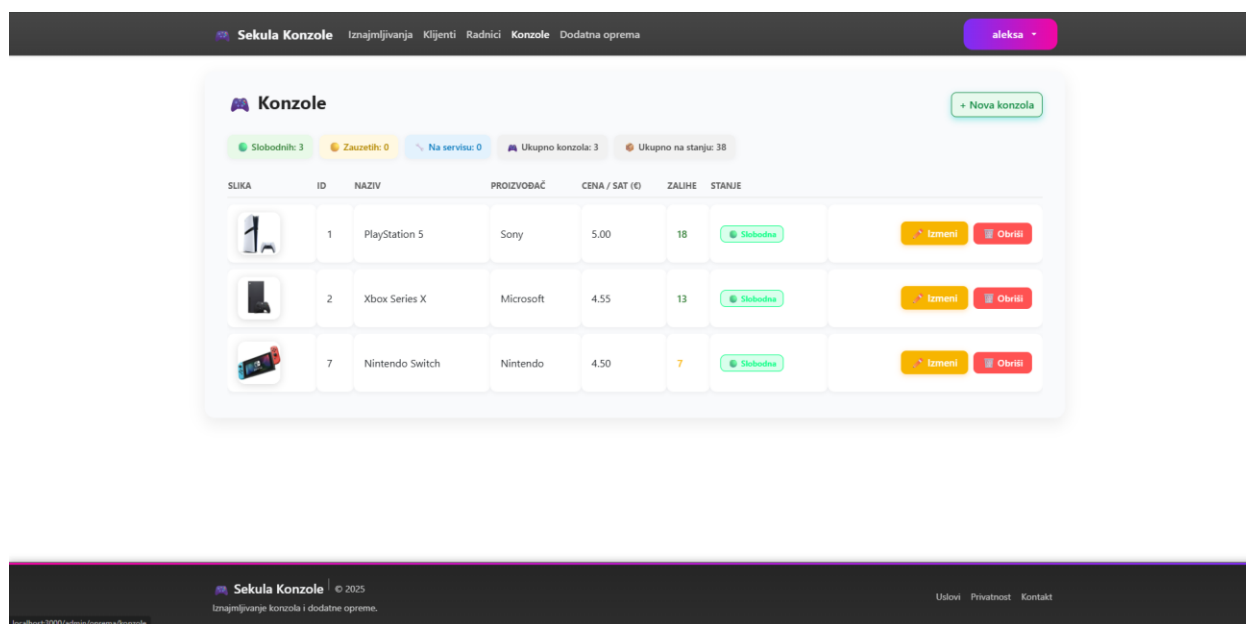
Радник

Учесници СК:

Радник, интерфејс, систем

Предуслови:

Радник је пријављен.



Слика 23: Форма приказ конзола

Основни сценарио СК:

1. Радник позива систем да отвори форму за унос конзоле. (АПСО)
2. Систем приказује форму са пољима (назив, произвођач, цена по сату, инвентарски број, стање). (ИА)
3. Радник уноси податке и потврђује. (АПУСО)

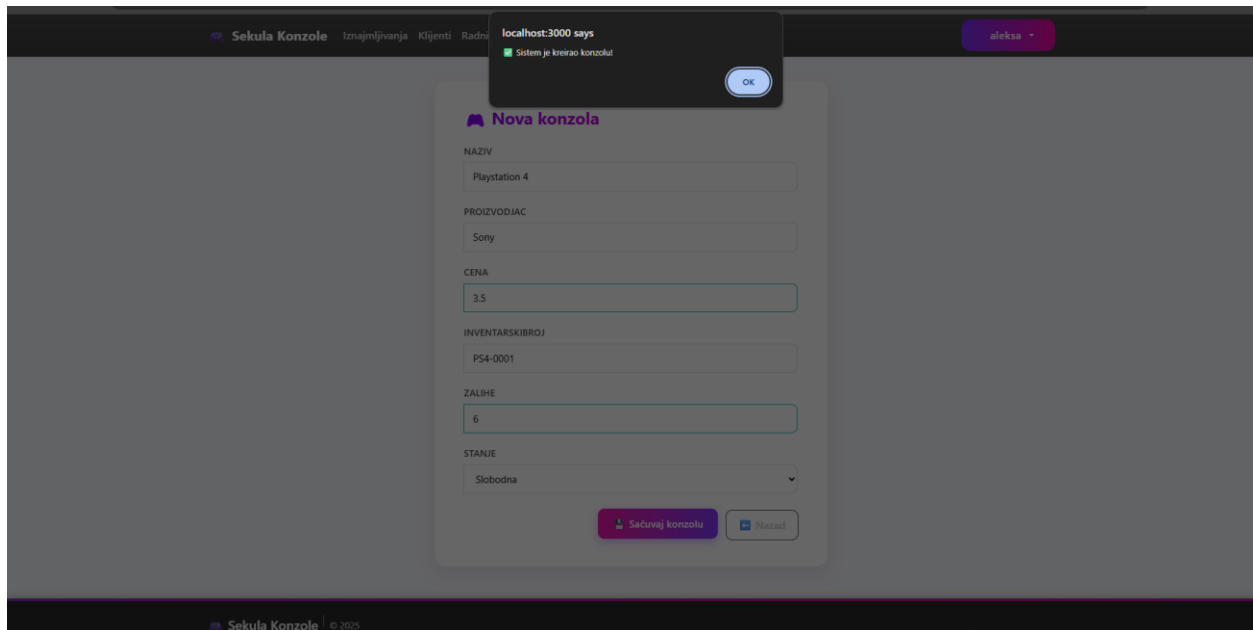
The screenshot shows a web application interface for 'Sekula Konzole'. At the top, there is a navigation bar with links: 'Iznajmljivanja', 'Klijenti', 'Radnici', 'Konzole', and 'Dodatna oprema'. A user profile 'aleksa' is visible in the top right. The main content area displays a form titled 'Nova konzola' with the following fields:

- NAZIV: Playstation 4
- PROIZVOĐJAC: Sony
- CENA: 3.5
- INVENTARSKIBROJ: PS4-0001
- ZALIHE: 6
- STANJE: Slobodna

At the bottom of the form, there are two buttons: 'Sačuvaj konzolu' (Save console) and 'Nazad' (Back). The footer of the page includes the 'Sekula Konzole' logo, copyright information '© 2025', and additional links: 'Ulogov', 'Dokument', and 'Kontakt'.

Слика 24: Приказ форме креирај конзолу са попуњеним подацима

4. Систем чува конзолу у бази. (CO)
5. Систем приказује поруку: „Sistem je kreirao konzolu!“ (ИА)



Слика 25: Порука успешног креирања конзоле

СК9 – Брисање конзоле

Назив СК:

Брисање конзоле

Актори СК:

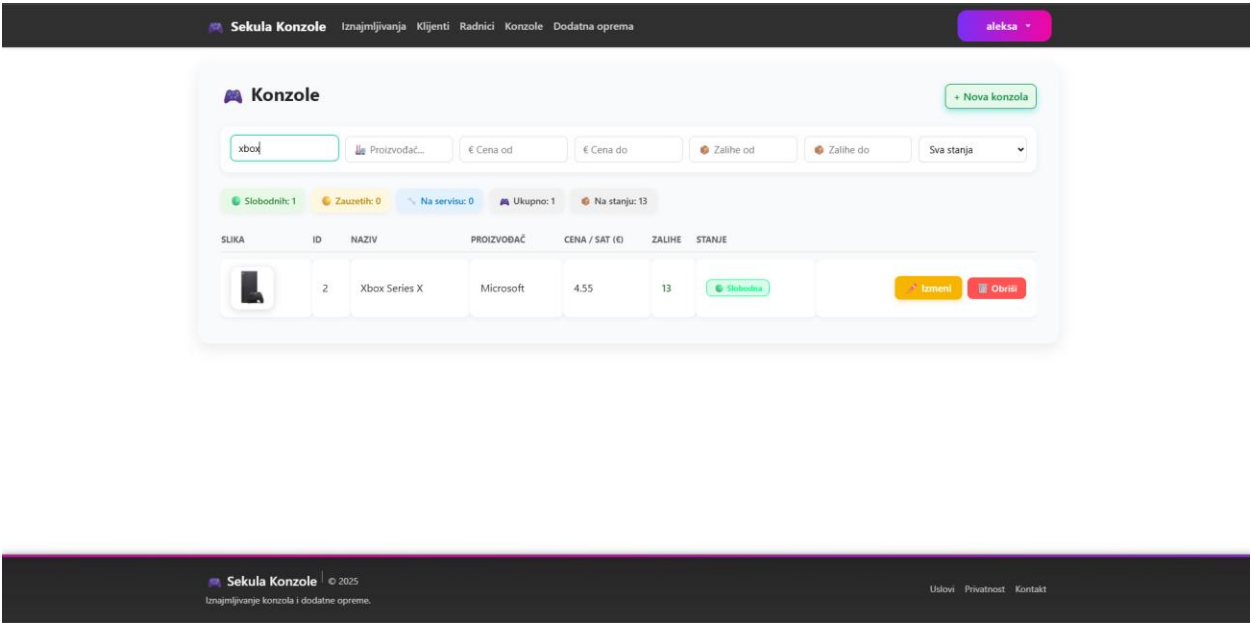
Радник

Учесници СК:

Радник, интерфејс, систем

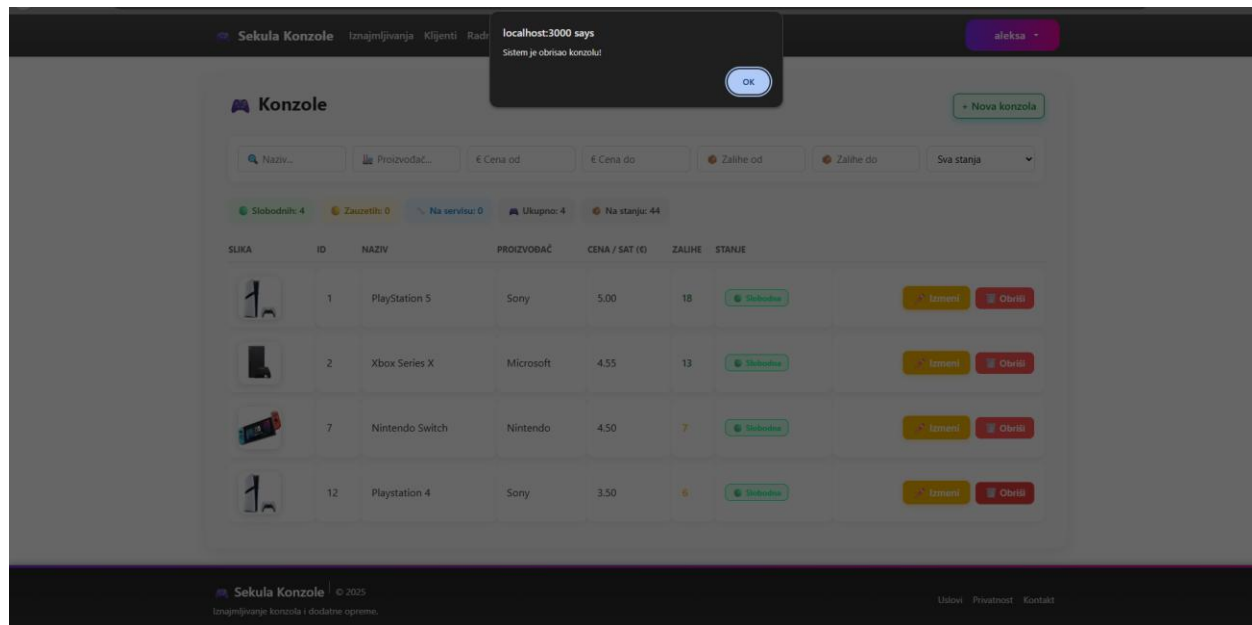
Основни сценарио СК:

- 1. Радник претражује конзоле. (АПУСО)
- 2. Систем приказује листу. (ИА)



Слика 26: Приказ успешног претраживања конзоле

3. Радник бира конзолу и позива систем за брисање. (АПСО)
4. Систем брише податке. (СО)
5. Систем приказује поруку: „Console successfully deleted!“ (ИА)



Слика 27: Приказ успешног брисања конзоле

СК10 – Претрага конзоле

Назив СК:

Претрага конзоле

Актори СК:

Клијент или радник

Учесници СК:

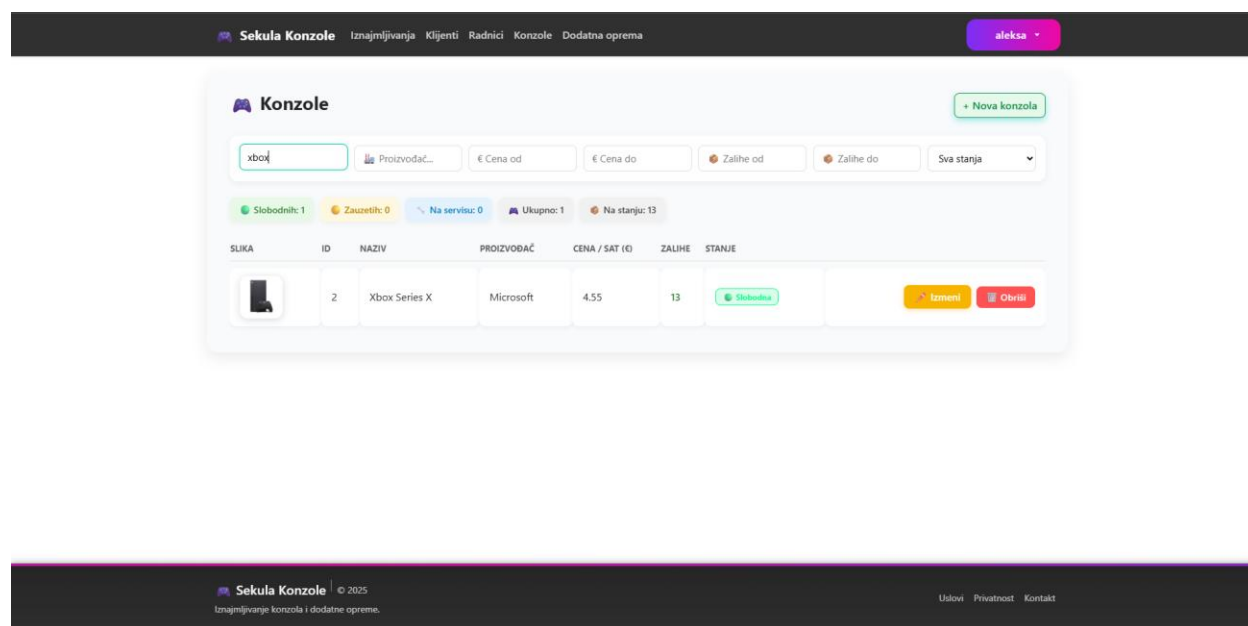
Корисник, интерфејс, систем

Предуслови:

Клијент и сервер су покренути.

Основни сценарио СК:

1. Корисник уноси параметре претраге (назив, произвођач, стање). (АПУСО)
2. Систем претражује базу података. (СО)
3. Систем приказује резултате у табели или картицама. (ИА)



Слика 28: Приказ успешног претраживања конзоле

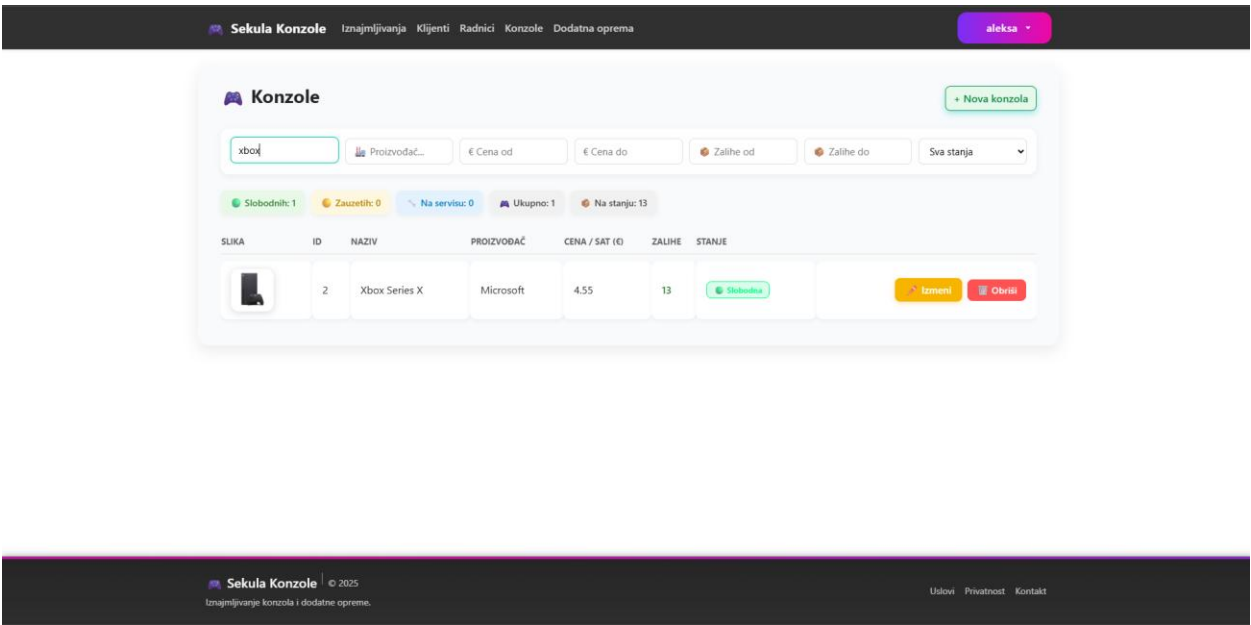
СК11 – Измена конзоле

Назив СК:
Измена конзоле

Актори СК:
Радник

Основни сценарио СК:

- 1. Радник претражује конзоле. (АПУСО)
- 2. Систем приказује резултате. (ИА)



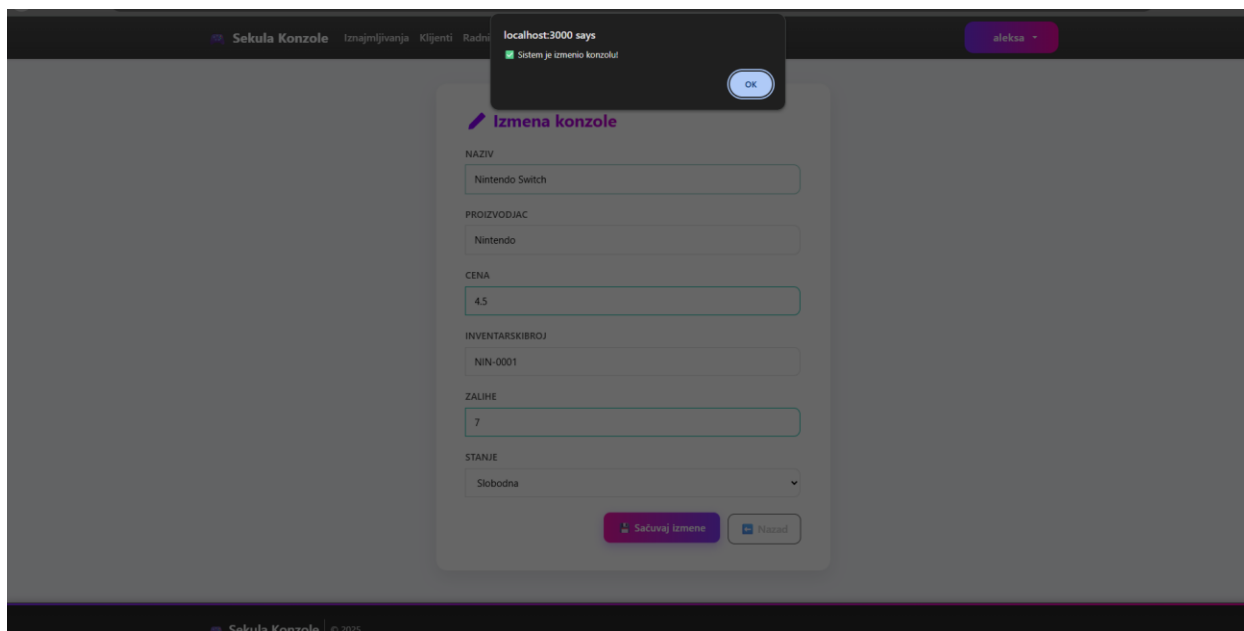
Слика 29: Приказ успешног претраживања конзоле

3. Радник мења податке (цена, произвођач, стање). (АПУСО)
4. Радник потврђује измене. (АПСО)

The screenshot shows a web application interface for 'Sekula Konzole'. At the top, there is a navigation bar with links: 'Iznajmljivanje', 'Klijenti', 'Radnici', 'Konzole', and 'Dodatna oprema'. A user profile 'aleksa' is visible in the top right. The main content area displays a form titled 'Izmena konzole' with a pencil icon. The form fields are: 'NAZIV' (Nintendo Switch Series 12 Pack 6), 'PROIZVODJAC' (Nintendo), 'CENA' (5), 'INVENTARSKIBROJ' (NIN-2221), 'ZALIHE' (7), and 'STANJE' (Slobodna). At the bottom of the form are two buttons: 'Sačuvaj izmene' (Save changes) and 'Nazad' (Back). The footer of the application shows 'Sekula Konzole | © 2025' and a status bar with 'Ukloni', 'Povrati', and 'Kontakt'.

Слика 30: Приказ попуњене форме измени конзолу

5. Систем чува податке. (СО)
6. Систем приказује поруку: „Console successfully updated!“ (ИА)



Слика 31: Приказ успешне измене конзоле

СК12 – Креирање изнајмљивања

Назив СК:

Креирање изнајмљивања

Актори СК:

Радник

Учесници СК:

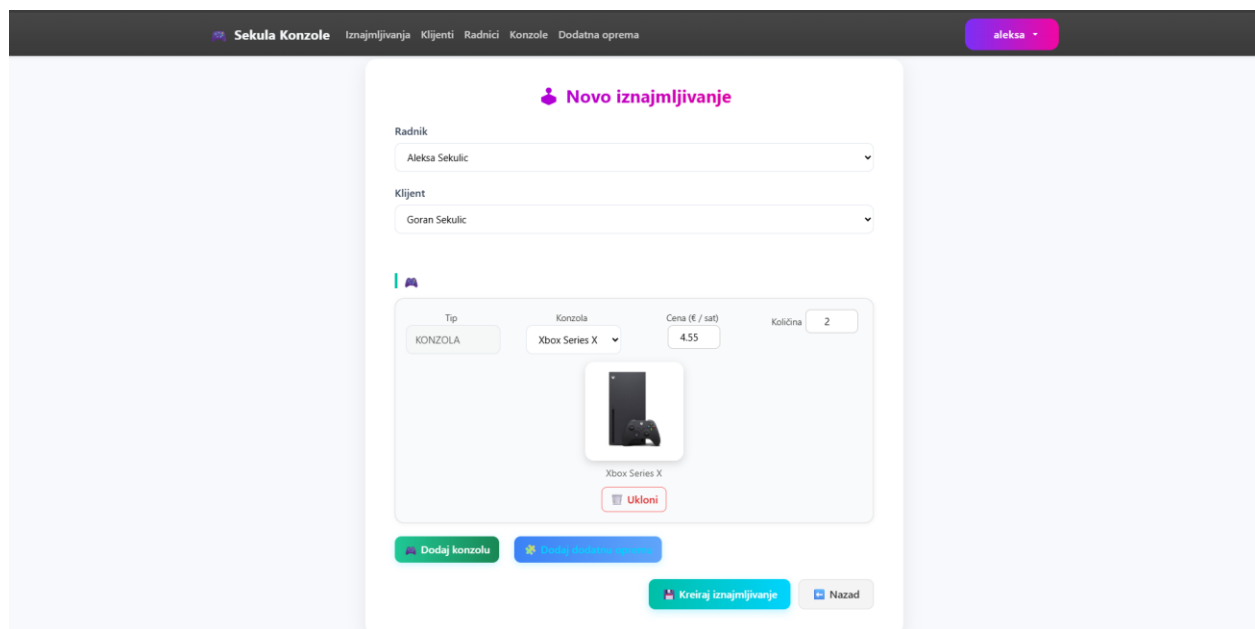
Корисник, интерфејс, систем

Предуслови:

Клијент и сервер су покренути, постоји бар једна доступна конзола (SLOBODNA).

Основни сценарио СК:

1. Радник бира конзолу. (АПУСО)



Sekula Konzole Iznajmljivanja Klijenti Radnici Konzole Dodatna oprema **aleksa**

Novo iznajmljivanje

Radnik
Aleksa Sekulic

Klijent
Goran Sekulic

Tip: KONZOLA Konzola: Xbox Series X Cena (€ / sat): 4.55 Količina: 2

Xbox Series X

Ukloni

Dodaj konzolu Novo iznajmljivanje Kreiraj iznajmljivanje Nazad

Слика 32: Приказ попуњене форме креирај изнајмљивање

2. Радник позива систем да креира изнајмљивање. (АПСО)
3. Систем повезује радника и конзолу. (СО)
4. Систем чува податке. (СО)
5. Систем приказује форму детаља за изнајмљивање и броји време трајања: „Време изнајмљивања:“ (ИА)

 Sekula Konzole

IznajmljivanjaKlijentiRadniciKonzoleDodatna oprema

aleksa

Iznajmljivanje #22

U_TOKU

Izmeni

KLIENT
Goran Sekulic

POČETAK
17.10.2025 12:52:54

KRAJ
-

STATUS
U_TOKU

UKUPAN IZNOS
0.00 €

Vreme iznajmljivanja:

00:00:11

Stavke iznajmljivanja



Xbox Series X

4.55 € / sat

Količina: 2

Vreme: 0 h

Iznos: 0.00 €

Слика 33: Приказ успешног креирања изнајмљивања

СК13 – Измена изнајмљивања

Назив СК:

Измена изнајмљивања

Актори СК:

Радник

Sekula KonzoleIznajmljivanjaKlijentiRadniciKonzoleDodatna oprema

aleksa

Iznajmljivanja

+ Novo

Pretraga po imenu klijentaID...mm/dd/yyyyStatus-- Filter po plaćanju --

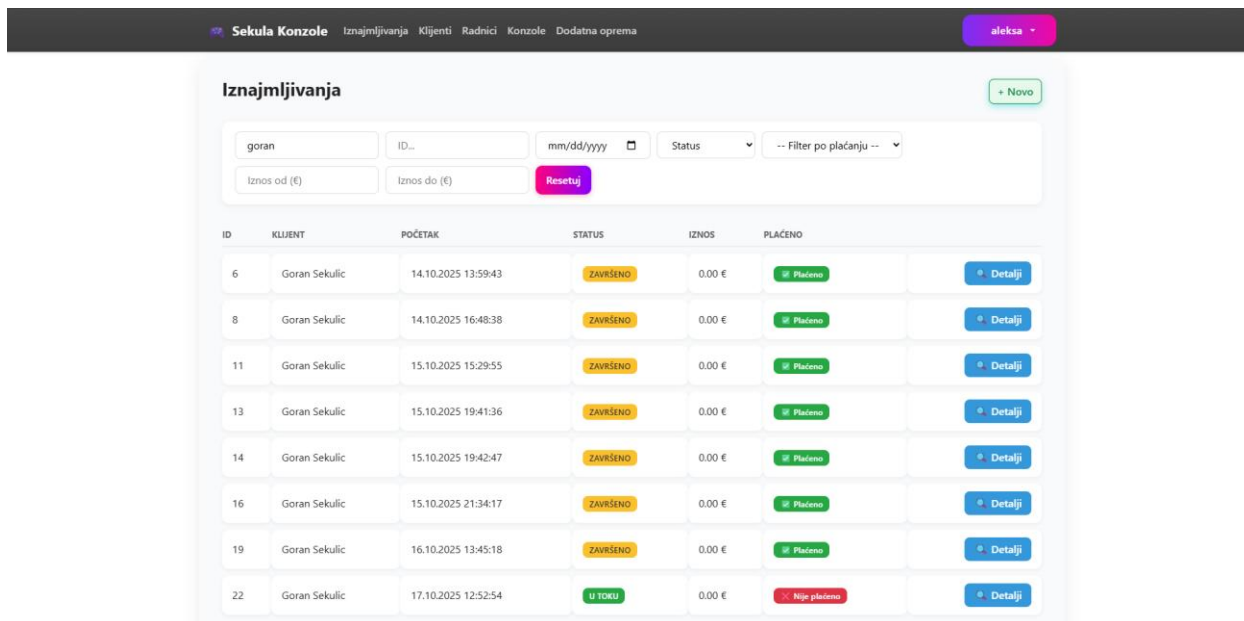
Iznos od (€)Iznos do (€)Resetuj

ID	KLIJENT	POČETAK	STATUS	IZNOS	PLAĆENO	
5	Mina Minić	14.10.2025 13:58:56	ZAVRŠENO	0.00 €	Nije plaćeno	Detalji
6	Goran Sekulic	14.10.2025 13:59:43	ZAVRŠENO	0.00 €	Plaćeno	Detalji
7	Mina Minić	14.10.2025 16:10:45	ZAVRŠENO	660.00 €	Plaćeno	Detalji
8	Goran Sekulic	14.10.2025 16:48:38	ZAVRŠENO	0.00 €	Plaćeno	Detalji
9	Mina Minić	15.10.2025 08:21:32	ZAVRŠENO	0.00 €	Plaćeno	Detalji
10	Pera Perić	15.10.2025 10:23:11	OTKAZANO	0.00 €	Nije plaćeno	Detalji
11	Goran Sekulic	15.10.2025 15:29:55	ZAVRŠENO	0.00 €	Plaćeno	Detalji
12	Mina Minić	15.10.2025 19:41:14	ZAVRŠENO	0.00 €	Nije plaćeno	Detalji

Слика 34: Форма приказ изнајмљивања

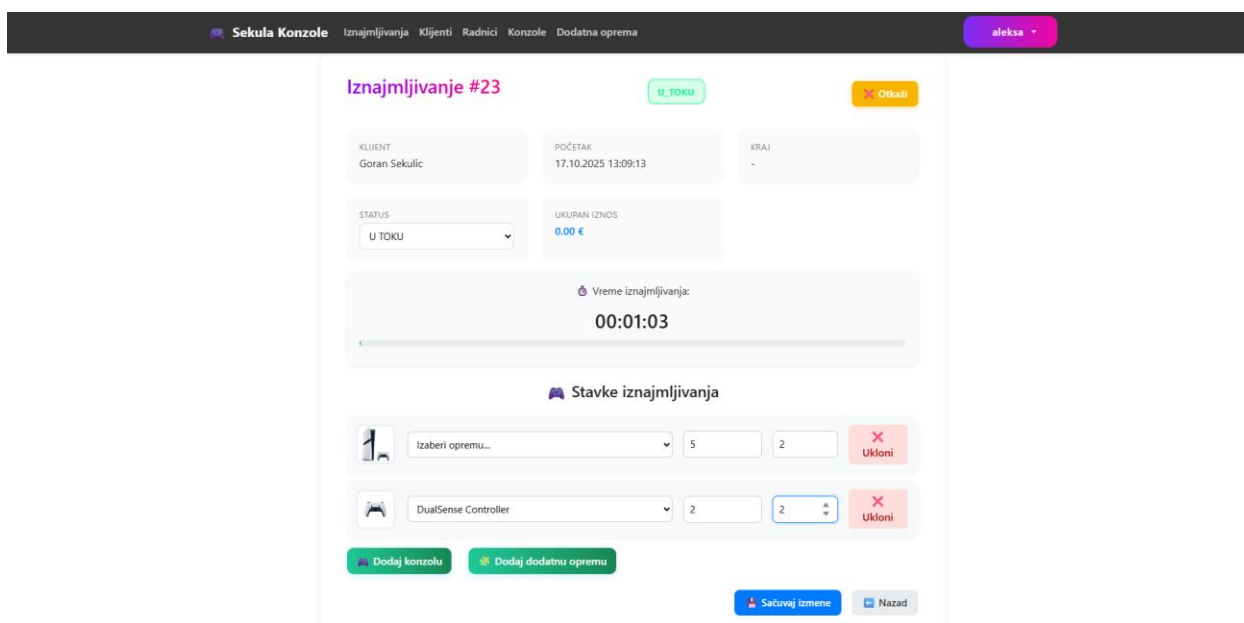
Основни сценарио СК:

1. Радник претражује изнајмљивања. (АПУСО)
2. Систем приказује резултате. (ИА)



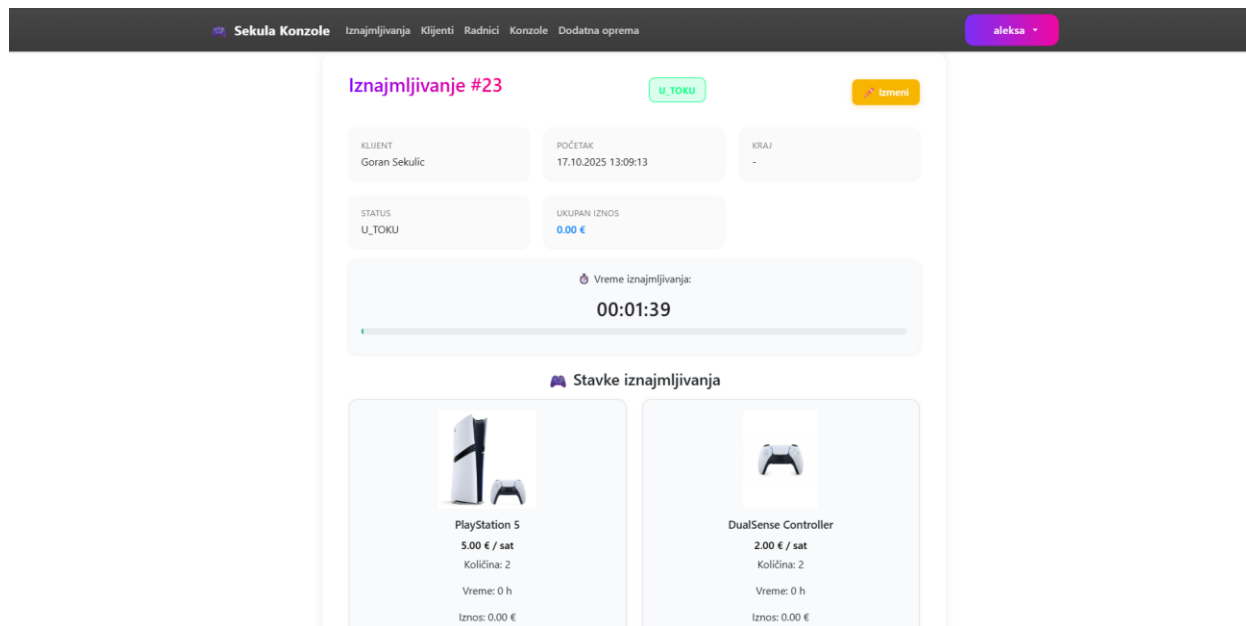
Слика 35: Приказ успешне претраге изнајмљивања

3. Радник мења ставке или статус. (АПУСО)
4. Радник потврђује измене. (АПСО)



Слика 36: Приказ измењеног изнајмљивања

5. Систем чува податке. (CO)
6. Систем приказује измењено изнајмљивање. (ИА)



Слика 37: Приказ измењеног изнајмљивања

СК14 – Претрага изнајмљивања

Назив СК:

Претрага изнајмљивања

Актори СК:

Радник

Учесници СК:

Радник, интерфејс, систем

Основни сценарио СК:

1. Радник уноси параметре претраге (клијент, датум, статус). (АПУСО)
2. Систем претражује базу изнајмљивања. (СО)
3. Систем приказује резултате у табели. (ИА)

Sekula Konzole

Iznajmljivanja

Klijenti

Radnici

Konzole

Dodatna oprema

aleksa

Iznajmljivanja

+ Novo

goran

ID..

mm/dd/yyyy

Status

-- Filter po plaćanju --

Iznos od (€)

Iznos do (€)

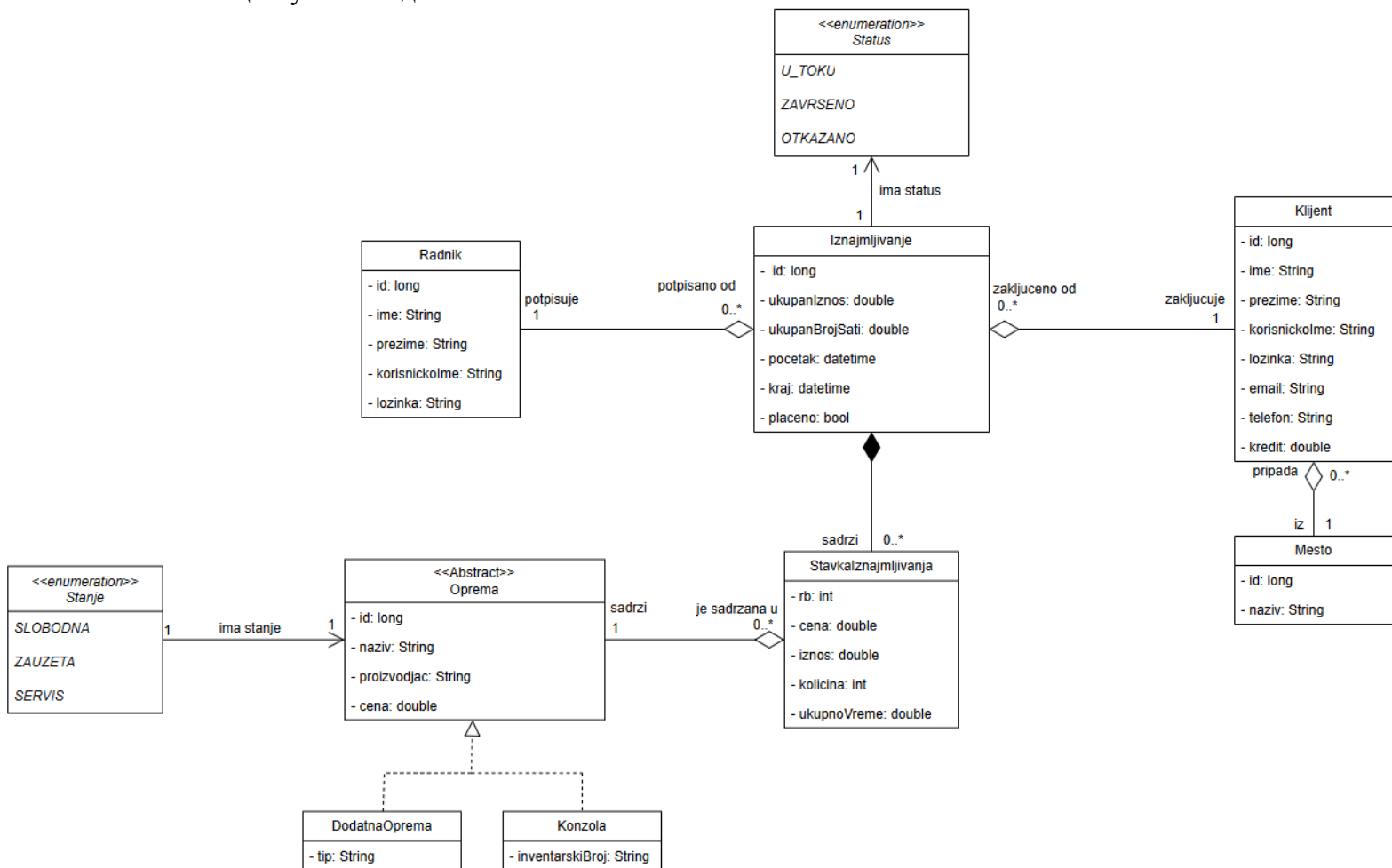
Resetuj

ID	KLIJENT	POČETAK	STATUS	IZNOS	PLAĆENO	
6	Goran Sekulic	14.10.2025 13:59:43	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
8	Goran Sekulic	14.10.2025 16:48:38	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
11	Goran Sekulic	15.10.2025 15:29:55	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
13	Goran Sekulic	15.10.2025 19:41:36	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
14	Goran Sekulic	15.10.2025 19:42:47	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
16	Goran Sekulic	15.10.2025 21:34:17	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
19	Goran Sekulic	16.10.2025 13:45:18	ZAVRŠENO	0.00 €	Plaćeno	+ Detalji
22	Goran Sekulic	17.10.2025 12:52:54	U TOKU	0.00 €	Nije plaćeno	+ Detalji

Слика 38: Приказ успешне претраге изнајмљивања

5. Структура пројекта

Концептуални модел:



Слика 39: Концептуални модел

На основу концептуалног модела направљен је следећи релациони модел:

1. **Radnik** (*idRadnik*, ime, prezime, korisnickoIme, lozinka)
2. **Mesto** (*idMesto*, naziv)
3. **Klijent** (*idKlijent*, ime, prezime, korisnickoIme, lozinka, email, telefon, kredit, *idMesto*)
4. **Iznajmljivanje** (*idIznajmljivanje*, ukupanIznos, ukupanBrojSati, pocetak, kraj, placeno, *idStatus*, *idKlijent*, *idRadnik*)
5. **Oprema** (*idOprema*, naziv, proizvođjac, cena)
6. **Konzola** (*idKonzola*, inventarskiBroj, *idOprema*)
7. **DodatnaOprema** (*idDodatnaOprema*, tip, *idOprema*)
8. **StavkaIznajmljivanja** (*id*, *rb*, cena, iznos, kolicina, ukupnoVreme, *idOprema*, *idIznajmljivanje*)

5.1 Структура система

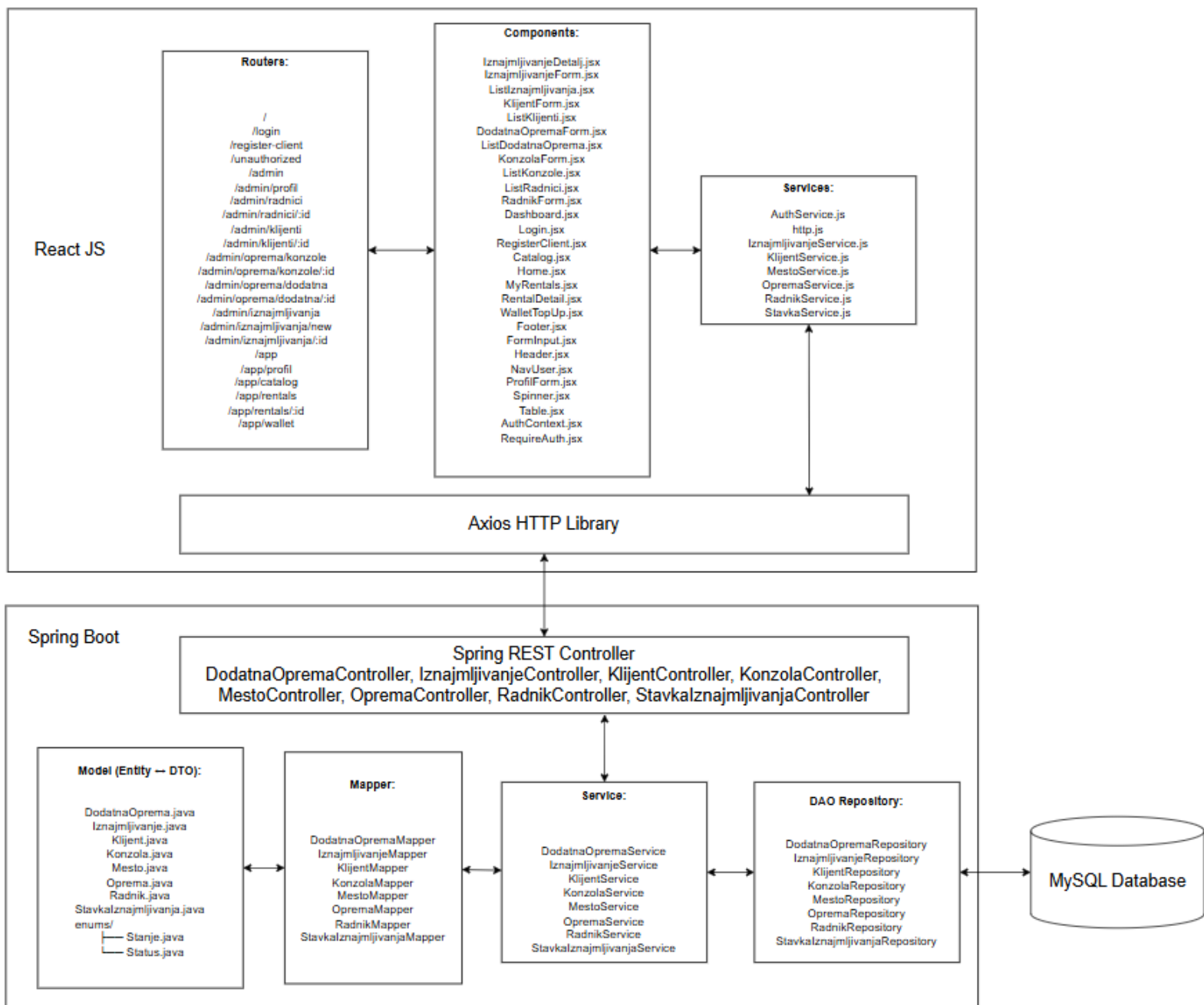
У React-у све почиње од компоненти које користе React сервисе да шаљу **REST** позиве преко **HTTP** клијента (**Axios** или **Fetch API**) ка **Spring Boot backend-у**.

Контролери у Spring Boot-у примају ове захтеве и прослеђују их сервис слоју који садржи пословну логику.

Сервис користи **JPA репозиторијуме** за приступ бази података и добија **Entity** објекте.

Када се подаци враћају према фронтенду, сервис користи **Mapper** да конвертује **Entity** → **DTO**, који контролер затим враћа React-у у **JSON** формату.

React сервис прима те податке, обрађује DTO и прослеђује их компонентама за приказ и манипулацију.



Слика 40: Архитектура система

На слици је приказана архитектура система која се заснива на трослојној структури. Frontend део апликације развијен је у React JS-у и садржи компоненте, руте и сервисе који преко Axios библиотеке шаљу REST захтеве ка backend-у. Backend део је имплементиран у Spring Boot оквиру и организован је у више логичких слојева: контролере (Controller), мапере (Mapper), сервисни слој (Service) и слој приступа подацима (DAO Repository).

Контролери примају и обрађују HTTP захтеве, сервисни слој реализује пословну логику, док репозиторијуми врше комуникацију са MySQL базом података. Mapper слој омогућава конверзију између ентитетских (Entity) и DTO објеката, чиме се обезбеђује јасна разграниченост између доменског и представљачког модела.

Описаћемо један пролаз кроз апликацију на примеру **додавања конзоле**.

Процес почиње од методе `addKonzola()` у React компоненти **KonzolaForm.jsx**, која се позива кликом на дугме „Додај конзолу“ на форми.

```
async function onSubmit(e) {
  e.preventDefault();
  setErr("");

  try {
    const cleanForm = {
      ...form,
      naziv: form.naziv.trim(),
      proizvođač: form.proizvođač.trim(),
      inventarskiBroj: form.inventarskiBroj.trim(),
      zalihe: Number(form.zalihe) || 0,
    };

    if (
      !form.naziv ||
      !form.proizvođač ||
      !form.cena ||
      !form.inventarskiBroj
    ) {
      alert("Sva polja su obavezna!");
      return;
    }

    if (isNew) {
      await OpremaService.createKonzola(cleanForm);
      alert("✅ Sistem je kreirao konzolu!");
    } else {
      await OpremaService.updateKonzola(id, cleanForm);
      alert("✅ Sistem je izmenio konzolu!");
    }

    nav("/admin/oprema/konzole");
  } catch (e) {
    setErr(e?.response?.data?.message || "Greška pri čuvanju podataka.");
  }
}
```

Корисник попуњава форму, а након клика на дугме, подаци се шаљу сервису `OpremaService` који преко `Axios`-а прави HTTP POST захтев ка backend-у.

```
export async function createKonzola(dto) { return (await api.post("/konzole", dto)).data; }
```

Овде се позива REST endpoint `/konzole` на Spring Boot backend-у, где се шаље објекат `konzola` у JSON формату.

```
@PostMapping
public ResponseEntity<KonzolaDto> createKonzola(@RequestBody KonzolaDto dto) {
    return new ResponseEntity<>(konzolaService.createKonzola(dto), HttpStatus.CREATED);
}
```

Контролер прихвата HTTP POST захтев, прослеђује DTO сервису и враћа поруку о успешности.

```
@Override
public KonzolaDto createKonzola(KonzolaDto dto) {
    return KonzolaMapper.mapToDto(repo.save(KonzolaMapper.mapToEntity(dto)));
}
```

Сервис користи **Mapper** да DTO објекат претвори у ентитет Konzola, а затим га чува у базу преко JPA репозиторијума.

```
public interface KonzolaRepository extends JpaRepository<Konzola, Long> {
}
```

Овде JPA обезбеђује CRUD операције.

```
public class KonzolaMapper {
    public static KonzolaDto mapToDto(Konzola k) {
        if (k == null) return null;

        KonzolaDto dto = new KonzolaDto();
        dto.setId(k.getId());
        dto.setNaziv(k.getNaziv());
        dto.setProizvodjac(k.getProizvodjac());
        dto.setCena(k.getCena());
        dto.setStanje(k.getStanje());
        dto.setZalihe(k.getZalihe());
        dto.setInventarskiBroj(k.getInventarskiBroj());
        return dto;
    }

    public static Konzola mapToEntity(KonzolaDto dto) {
        if (dto == null) return null;
        Konzola k = new Konzola();
        k.setId(dto.getId());
        k.setNaziv(dto.getNaziv());
        k.setProizvodjac(dto.getProizvodjac());
        k.setCena(dto.getCena());
        k.setStanje(dto.getStanje());
        k.setZalihe(dto.getZalihe());
        k.setInventarskiBroj(dto.getInventarskiBroj());
        return k;
    }
}
```

Mapper служи за претварање података између DTO објекта који долази из frontend-a и ентитета који се чува у бази података.

Цео ток рада изгледа овако:

**KonzolaForm.jsx → OpremaService.js → KonzolaController.java → KonzolaService.java
→ KonzolaRepository.java → MySQL база**

Овим је покривен комплетан процес креирања нове конзоле у систему, од клика на дугме у React форми до уписа новог реда у базу.

6. Закључак

Закључак овог пројекта наглашава значај употребе савремених технологија као што су **Spring Boot** и **React.js** у развоју модерних веб апликација. За разлику од традиционалних решења заснованих искључиво на десктоп окружењима и класичним Java GUI библиотекама, комбинација ових технологија доноси потпуну интеграцију између backend-a и frontend-a, уз висок ниво интерактивности, брзине и прилагодљивости.

Spring Boot представља поуздан и скалабилан оквир за изградњу backend система. Он омогућава брзу имплементацију REST сервиса, ефикасну комуникацију са базом података преко **JPA** и једноставно управљање зависностима уз помоћ **Maven-a**.

На страни корисничког интерфејса, **React.js** обезбеђује динамичан и модеран frontend који омогућава интуитивну навигацију, лако одржавање компоненти и асинхрону комуникацију са сервером преко **Axios HTTP библиотеке**. Овај приступ чини апликацију пријатном за коришћење, визуелно прегледном и одговарајућом очекивањима савременог корисника.

Комбинација **React-a**, **Spring Boot-a** и **MySQL базе** показала се као изузетно стабилна архитектура која подржава даљи развој и унапређење система. Пројекат је показао да овај технолошки стек представља идеално решење за примену у реалним пословним апликацијама, као што је систем за изнајмљивање видео-конзола, где су потребни поузданост, сигурност и једноставна надоградња функционалности у будућности.